

IOT ANTI-SLEEP GLASSES

Milestone 1: Planning and design



Contents

Project plan	5
1. Executive Summary	5
2. Project Scope	5
2.1 In scope:	5
2.2 Out of scope:	5
3. Deliverables	6
3.1 Milestone 1 deliverables:	6
3.2 Milestone 2 deliverables:	6
4. Background and motivation	6
4.1 Why anti-sleep glasses are needed	7
5. Stakeholders	8
6. Expected benefits	8
7. Literature review	8
7.1 Overview of existing technologies	9
7.2 Comparative analysis	10
7.3 Gaps in Current Fatigue Detection Systems	10
7.4 How the project improves on existing solutions	11
8. Requirements specifications	12
9. Risk Assessment and Mitigation Plan	13
10. Project Implementation Plan and Timeline	14
11. The Team	15
12. Required Hardware, Software and Tools	16
12.1. Hardware with prices	16
12.2. AI and Machine Learning Tools	17
12.3. Connectivity and Integration	17
12.4. Mobile and Dashboard Development	17
12.5. Testing and Development Tools	17
12.6. Design and Simulation Tools	17
Project Design	18
1. Hardware Specifications	18
1.1 Core components	18
1.2 Hardware Architecture for Anti-Sleep Glasses	18

1.3 Hardware Workflow	23
1.4 Error Handling and Fault Tolerance.....	24
1.5 Hardware Development Plan	24
1.6 Scalability and Future Enhancements	24
1.7 Connectivity and Pin Assignments	25
1.8 Calculate Total Power Consumption	31
1.9 Power Optimization Strategy	34
1.10 Circuit Diagram.....	35
1.11 How it works	35
1.12 Diagram	36
2. Software Design.....	37
2.1 Software Architecture	37
2.2. Software Workflow.....	39
2.3. Error Handling and Fault Tolerance.....	39
2.4 Tech Stack and Tools.....	40
2.5. Software Development Plan	41
2.6 Scalability and Future Enhancements	42
2.7. Software flow	43
3. User Experience and User Design	44
3.1. Introduction	44
3.2. Objectives.....	44
3.3. User Personas and Use Cases	44
3.4. Alert System Interface Design	45
3.5. Alert Workflow	47
3.6. User Interface Design.....	47
3.7. Purpose and Functionality.....	48
3.8. Mobile App Interface	48
3.9. Web Dashboard Interface.....	49
3.10. Car Infotainment Integration	50
3.11. Accessibility and Inclusivity	50
3.12. Development Plan	50
3.13. Future Enhancements	51
3.14. Summary	51

3.15 Sample of UI dashboard on Chart.js	52
3.16 hart.js mock-up dashboard	52
4. AI/ML model for anti-drowsiness	53
4.1. Introduction	53
4.2. Objectives	53
4.3. Data Collection	53
4.4. Model Architecture	56
4.5. Training and Evaluation	58
4.6. Integration.....	60
4.7. Future Enhancements.....	61
4.8. Summary	61
5. Preliminary Testing Plan for Anti-Sleep Glasses	62
5.1. Introduction	62
5.2. Testing Objectives.....	62
5.3. Testing Plan	63
5.4. Quality Benchmarks	66
5.5. Testing Protocols	66
5.6. Metrics for Success	67
5.7. Metrics for Sustainability.....	68
5.8. Strategies.....	68
5.9. Test Environments	68
5.10. Timeline Alignment	69
6. Glasses Frame Design	70
Extra information	73
What is driver fatigue/drowsiness and why is it important to detect it?	73
Will light sources increase a person's awareness	73
Does this product already exist and how is it used.....	74
Will sunglasses or normal glasses construction be efficient enough for anti-sleep glasses	75
How long until effective comfortability is diminished for having glasses on	75
What expert recommendations or guidelines exist for managing or detecting fatigue?	76
What are the current research trends shaping the future of fatigue detection systems?	77

What technologies or methods are currently used to detect fatigue?	77
References	79

Project plan

1. Executive Summary

This project introduces an Anti-Sleep glasses system. The aim for the glasses is to detect signs of fatigue within a driver through the monitoring of eye-blinks and head movements in real-time. The system integrates an Infrared sensor, an MPU6050 motion sensor and an ESP32 microcontroller to process input and trigger immediate feedback. The system makes use of AI and Machine Learning to process the given input, and trigger immediate feedback through an LED light, buzzer or vibration motor. These glasses are designed to be lightweight, comfortable and suitable for long-term use, especially when driving.

The purpose of this project is to reduce fatigue related incidents on the road, particularly for long-distance drivers, transportation professionals and shift-based industries. By doing so, roads will be safer, leading to less accidents. The goal is to develop an affordable, portable and effective warning system that will provide immediate alerts when drowsiness is detected.

2. Project Scope

The scope of the project defines the objectives and functions that will be addressed in the design and implementation of the system. The system aims to track driver fatigue using sensors, artificial intelligence and machine learning, and provide immediate alerts. To ensure the successful completion of the project, it is important to outline what will be included in the scope, as well as what will fall outside of the scope.

2.1 In scope:

- Eye blink tracking using infrared sensors
- An AI model for drowsiness detection
- Bluetooth/Wi-Fi connectivity to car dashboard for alerts
- Buzzer and vibration alerts in glasses for feedback

2.2 Out of scope:

- Fully autonomous vehicle control (system only provides alerts)
- Medical grade EEG brainwave monitoring
- Direct vehicle integration (braking, lane control, etc.)

3. Deliverables

The deliverables that are outlined below represent the key outputs that will be produced throughout the course of the project. These outcomes serve as milestones for tracking progress, and ensuring all requirements are met.

3.1 Milestone 1 deliverables:

- Project concept
- System planning and design
- Hardware selection

3.2 Milestone 2 deliverables:

- Implementation of system design
- Fully functioning anti-sleep glasses

4. Background and motivation

Falling asleep at the wheel is a serious problem that can cause many accidents. When drivers become too tired and fall asleep, they can easily lose control of their vehicle, which can lead to dangerous or even fatal crashes. This is especially risky during long distance drives or late at night when people naturally become more tired. These types of fatigue-related incidents are a major cause of road accidents, and even a few seconds of sleep can have serious consequences, as reported by (Heliyon, 2024).

That's where the Anti-sleep glasses system comes in. It is designed to catch the early signs of drowsiness before anything dangerous can happen. According to (PhilArchive, n.d.), the glasses monitor how long the user's eyes stay closed. If the eyes remain shut for more than three seconds, the system activates a buzzer to wake the user. This kind of fast alert gives drivers a chance to regain focus and avoid potential accidents (PhilArchive, n.d.).

The overall goal is to prevent crashes before they occur. As explained by (PhilArchive, n.d.), the glasses work in real time to spot signs of tiredness and respond immediately, helping drivers stay alert and keeping the roads safer for everyone.

Falling asleep while driving continues to be one of the most dangerous issues on the road. Many accidents happen when people drive for long hours or during late shifts. Even a momentary loss of awareness can result in a crash. As stated by (Heliyon, 2024), anti-sleep glasses are being developed specifically to tackle this issue by detecting signs of fatigue early and prompting drivers to stay awake.

Professional drivers are especially at risk, as noted in (Heliyon, 2024). The most worrying part is that many people do not realise they're falling asleep until it's too late. Research shows that fatigue-related drowsiness remains one of the top causes of preventable accidents, and that current safety tools have not yet fully addressed the problem.

Older devices created to keep drivers awake often fell short. As discussed in (Heliyon, 2024), these devices were sometimes too large, uncomfortable, or complex to use. They could block the user's vision, restrict movement, or require frequent charging and bulky batteries. Many relied on lights or loud sounds to wake the user methods that don't always work when someone is truly exhausted.

That's why newer solutions are being explored. According to (PhilArchive, n.d.), today's anti-sleep glasses use eye tracking technology to detect fatigue in real time. Instead of loud alarms or flashing lights, they send a subtle but effective alert like a buzzer or vibration directly through the glasses to gently wake the driver without distraction.

As highlighted by (Heliyon, 2024), the field of fatigue detection is growing quickly. While new technologies are being tested, there are still gaps that need attention. Devices like these glasses help fill that gap by offering practical, real-world support to drivers.

At the end of the day, it all boils down to safety. If a simple wearable tool like this can help prevent even one accident, then it's a step in the right direction (PhilArchive, n.d.).

4.1 Why anti-sleep glasses are needed

Falling asleep while driving is not just risky, it is deadly. The National Highway Traffic Safety Administration reported that drowsy driving caused approximately 91,000 crashes in just one year (2017), leading to approximately 50,000 injuries and 800 deaths. These numbers highlight why we need better ways to keep drivers awake and alert, according to (Arakawa, 2021).

However, older systems designed to detect drowsiness have flaws. They usually track things such as how the car is moving, heart rate, or images of your face. However, these methods do not always perform satisfactorily. Sometimes it is too dark, or the driver is wearing glasses, or they have conditions such as sleep apnea, these are things that diminish accuracy, stated by (Arakawa, 2021). Even when these systems operate, they often miss early warning signs, and this delay can make all the differences in preventing a crash.

Falling asleep while driving is not just risky, it is deadly. The National Highway Traffic Safety Administration reported that drowsy driving caused approximately 91,000 crashes in just one year (2017), leading to approximately 50,000 injuries and 800 deaths. These numbers highlight why we need better ways to keep drivers awake and alert, according to (Arakawa, 2021).

However, older systems designed to detect drowsiness have flaws. They usually track things such as how the car is moving, heart rate, or images of your face. However, these methods do not always perform satisfactorily. Sometimes it is too dark, or the driver is wearing glasses, or they have conditions such as sleep apnea, these are things that diminish accuracy, stated by (Arakawa, 2021). Even when these systems operate, they often miss early warning signs, and this delay can make all the differences in preventing a crash.

5. Stakeholders

Stakeholders include all of the people that fall part of the project. From the team developing the system, to the users of the system. The following groups are the key stakeholders:

- Our primary stakeholders include the users of the system.
- Our internal stakeholders consist of our development team.
- Our academic stakeholders consist of our instructors and supervisors
- Our external stakeholders include the industry that our system falls a part of.
- The beneficiaries of our system include the public and society.

6. Expected benefits

With the anti-sleep glasses, drivers can expect to become safer and more aware on the road. Noticing drowsiness early is a major feature which keeps the driver safe by preventing fatigue-caused slow reactions and less cautious decisions. If the driver looks tired, the system warns them promptly to reduce the risk of micro-sleeps which sometimes cause accidents. Because of these alerts, drivers become more cautious about their fatigue which motivates them to stop when necessary. The glasses were designed to be comfortable and unnoticeable when you wear them. They provide different ways to receive feedback, including custom options for lights, vibrations or a buzzer. Besides their use in driving, these glasses can help in workplaces, evening shifts and actions where staying awake is necessary. The system helps keep drivers alert on the road, making the roads safer and lessening chances of fatigue-related accidents.

7. Literature review

This literature review looks at published studies, tools and technologies used for fatigue detection with anti-sleep wearables. Since driver drowsiness is one of the major causes of road accidents, many are interested in systems that don't need direct contact to detect fatigue and stop accidents from occurring. The aim of this review is to look at fatigue detection technologies in use, such as AI-based wearables, sensor integration and platforms in vehicles or on smartphones. By analysing research, this review provides a solid base for creating and executing a new anti-sleep glasses system. Based on the review, major design decisions can be made, and development challenges and possible benefits will be known.

7.1 Overview of existing technologies

A rise in concern over drowsiness-induced accidents has led to the creation of numerous real-time drowsiness detection and tracking technologies. ESP32-CAM-based glasses are one of the most promising technologies that combine light hardware and AI capability. The systems read blinking patterns and identify indices of fatigue. Using a case study, Ms. Keerthana T. S. (2023) illustrates that the glasses identify onset drowsiness and notify the subject within 300 milliseconds, an instantaneous response that can prevent dangerous micro-sleep accidents.

Alongside camera modules, infrared sensors have been discovered to be of further utility in applications related to fatigue state monitoring. These sensors track eyelid movement and blink rate and are often paired with simple alert devices like buzzers or vibration motors. The process gives real-time feedback through a cheap and non-invasive method, states K. Nirosha (2024).

Beyond wearable glasses, technologies for detecting fatigue are now being applied to broader applications for occupational safety. Physiological signal monitoring by devices that monitor biomedical signals like ECG, for example, has been one of the effective measures for detecting symptoms of fatigue. According to Martins et al. (2021), these kinds of systems are best deployed for continuous tracking under critical situations since their intrusiveness is minimal.

Additionally, Moshawrab (2022) describes the application of smart wearables in an industrial setting where vital signs are monitored and analysed by AI models for detecting worker fatigue, opening up avenues for improved workplace safety. Driver drowsiness detection has been included within advanced driver-assistance systems in the automotive industry. They tend to utilize camera and eye-tracking sensors to monitor visual behaviour such as blink rate, yawning, and gaze shift. Wessel (2022) describes how such systems are capable of alerting drivers when there are indications of driver drowsiness and are asked to take a break. Some systems, such as those developed by Bosch, utilize steering behaviour analysis. Bosch's system measures up to 70 different inputs from the steering wheel to assess the alert state of a driver (Bosch, n.d.), illustrating how behaviour analytics are complemented by visual observation.

Technologically, AI and machine learning are now being applied to deliver greater precision and accuracy to fatigue detection systems. Multimodal data processing has been an area of recent interest, where inputs from spatially distributed sensors and cameras are fed into the system to increase detection reliability. According to Kakhi (2024), such combined approaches put models in a better position to identify subtle patterns of fatigue. Moreover, deep learning models such as YOLOv5- and vision transformer-based models have demonstrated best-in-class performance levels when detecting fatigue cues from real-time video streams, which is a critical area of intelligent transportation technology (arXiv, 2022).

Overall, the literature presents a clear way forward to design smarter, faster, and more accurate systems for detecting fatigue. Wearable or embedded within car systems, technologies share one single goal, that is improving safety through early and accurate detection of drowsiness.

7.2 Comparative analysis

Research on anti-sleep glasses is still very limited. Most existing literature focuses on individual prototypes or concepts rather than multiple comparative studies. However, examining how these glasses are built and how they function offers valuable insight into what sets certain designs apart especially when applied to real-time driver drowsiness detection, stated by (Shinganapure, 2022).

7.3 Gaps in Current Fatigue Detection Systems

Despite growing interest in fatigue monitoring, most current systems still have major limitations, especially in high-risk industries like cargo transport, logistics, and public transit. As noted by the (American journal of industrial medicine, 2022), there are several key gaps that need to be addressed to make these systems more effective and reliable in real-world settings.

First, there's the issue of real-time accuracy. Many current tools fail to catch the early warning signs of fatigue. They rely on limited input data and only respond once performance begins to drop. A more advanced approach, such as an Anti-sleep IoT system, could improve this by processing multiple data sources at once and delivering alerts before fatigue reaches a critical point, according to (American journal of industrial medicine, 2022).

Second, predictive capabilities are often lacking. Tools like electronic logging devices (ELDs) track how long someone has been driving, but they do not take individual fatigue levels into account, stated by (American journal of industrial medicine, 2022). By using AI and machine learning, smarter systems could predict when someone is likely to become too tired and suggest break times or schedule changes before it becomes a safety risk, according to (American journal of industrial medicine, 2022).

Another gap is the lack of strong feedback loops. Most systems alert the driver, but they do not provide actionable next steps. Ideally, a fatigue monitoring system should also notify marine managers, offer alternative routes, or help schedule immediate rest periods turning a simple alert into a complete response, stated by (American journal of industrial medicine, 2022).

There is also a challenge with integration into broader fatigue risk management strategies. As companies shift from strict hour-based models to more flexible approaches, existing systems often fail to provide the consistent, meaningful data needed for full integration, according to (American journal of industrial medicine, 2022). An Anti-sleep IoT platform could offer detailed analytics and performance metrics that support this transition.

The range of fatigue indicators being tracked is also too narrow. Most current tools only monitor one or two factors, such as steering input or blink frequency. But fatigue can come from a combination of things like sleep loss, emotional stress, heat, or vibration, stated by (American journal of industrial medicine, 2022). A more complete system would use multiple sensors to capture all of these influences and build a fuller picture of driver fatigue.

Lastly, even when data is collected, it is often not detailed enough to guide policy or shift planning. According to (American journal of industrial medicine, 2022), companies struggle to turn the data they have into practical decisions. A well-designed system could close that gap by producing clear, useful reports that help improve working conditions and safety rules.

All of these issues show us that the need for next-generation solutions that go beyond basic fatigue detection. What is needed now is a smarter, more adaptive approach that predicts fatigue, responds in real time, and supports both individual users and broader safety systems, stated by (American journal of industrial medicine, 2022).

7.4 How the project improves on existing solutions

Many existing anti-sleep devices rely on complex or expensive technologies, such as EEG sensors or high-resolution cameras. These systems are often bulky, require calibration, and can be impractical for everyday use. According to (American journal of industrial medicine, 2022), such advanced systems may not always be feasible for widespread application due to cost and operational complexity.

Our project takes a different approach one that emphasises simplicity, affordability, and real-world functionality. Key Improvements can be made, according to (American journal of industrial medicine, 2022).

The system utilizes affordable and widely available components, including an IR sensor, Arduino Pro Mini, and basic alert devices. This significantly reduces the overall cost and makes the technology more accessible to individual drivers or smaller transport companies. As noted by the (American journal of industrial medicine, 2022), cost-effective interventions are more likely to be adopted in practical settings. By embedding the IR sensor directly into the frame of a pair of glasses, the design also remains lightweight and comfortable for long-term wear avoiding the bulkiness often associated with camera-based or EEG headsets, stated by (American journal of industrial medicine, 2022). This design consideration aligns with findings from the same journal, which emphasise the importance of comfort and wearability in fatigue-monitoring tools.

In addition to its affordability and comfort, the system offers flexibility through the Arduino platform, allowing users to adjust the code to meet their specific needs. For example, they can fine-tune sensor sensitivity, change the duration of

alerts, or add new features depending on their environment or personal preferences, according to (American journal of industrial medicine, 2022). This adaptability makes the system versatile and practical for a wide range of users. Moreover, the IR-based design eliminates the need for complex calibration or setup, requiring minimal user input and no internet connectivity or mobile app syncing. This simplifies usage and makes it more reliable for drivers on the move.

To ensure clear and immediate feedback, the device incorporates both a buzzer and a vibration motor. These dual alerts are particularly effective for users who may become desensitized to sound alone. As suggested by research in the (American journal of industrial medicine, 2022), using multiple sensory cues can improve the responsiveness of fatigue detection systems. Unlike many commercial options that include unnecessary features, this design focuses solely on the essential function of keeping drivers awake and alert. By eliminating distractions and focusing on core functionality, the system increases reliability and effectiveness.

While research into anti-sleep glasses is still evolving, this project offers a practical and informed alternative that addresses many of the limitations found in existing fatigue detection systems. Drawing on insights from recent studies, including those in the (American journal of industrial medicine, 2022), it delivers a lightweight, cost-efficient, and customizable solution aimed at improving driver safety in real-world conditions.

8. Requirements specifications

- The system must make use of infrared-sensors to detect eye-blinks.
- The system must make use of AI and ML to process the data given by the sensors.
- The system must give alerts to the user when fatigue is detected in real-time through the use of a light, vibration and buzzer.
- The system must link to a car dashboard system to provide audible alerts.
- A mobile app must be developed for storing and processing user data.

9. Risk Assessment and Mitigation Plan

Risk	Likelihood	Impact	Mitigation Strategy
Component failure (IR sensor, MPU6050, etc.)	Medium	High	Order spares, test all components early, ensure compatibility.
Software bugs or model misclassification	High	High	Conduct unit tests, implement a structured testing framework, use real-world scenarios.
Delay in parts delivery	Medium	Medium	Order components in Week 1, identify alternate suppliers in Gauteng.
Battery runtime too short	Medium	High	Optimize power consumption, consider larger capacity or backup batteries.
Test ESP32's built-in BLE/Wi-Fi early, implement fallback to local CNN inference on ESP32 if connectivity fails.	Medium	Medium	Test both modes early, implement fallback communication protocols.
Incorrect alert triggering	Medium	High	Refine ML models through testing, set conservative thresholds in early deployment.
Integration with mobile app or dashboard	Medium	Medium	Assign a dedicated dev for communication protocols, start integration early.
Hardware discomfort (fit, heat, vibration)	Low	Medium	Collect user feedback, iterate on frame design, test for extended wear.
Team delays or coordination issues	Low	Medium	Weekly check-ins, shared progress tracker, early identification of blockers.

10. Project Implementation Plan and Timeline

Week(s)	Task
Week 1 (Jul 7–13)	Order and test hardware components. Finalize ML model structure.
Week 2 (Jul 14–20)	Begin an initial assembly of glasses. Set up ESP32 and basic sensor tests.
Week 3 (Jul 21–27)	Begin ML model training with real-world data. Test MPU6050 and IR sensor.
Week 4 (Jul 28–Aug 3)	Integrate ML model into ESP32 or Raspberry Pi. Initial software testing.
Week 5 (Aug 4–10)	Develop mobile app prototypes. Build early Bluetooth connection pipeline.
Week 6 (Aug 11–17)	Connect ESP32 to phone + dashboard (basic alerts).
Week 7 (Aug 18–24)	Begin structured testing (lab + simulation). Collect feedback.
Week 8 (Aug 25–31)	Adjust model, fix bugs, improve frame ergonomics.
Week 9 (Sep 1–7)	Finalize mobile app UI, connect to website dashboard.
Week 10 (Sep 8–14)	Conduct field testing: real driving simulations with alerts.
Week 11 (Sep 15–21)	Analyze field test data. Optimize latency and reliability.
Week 12 (Sep 22–28)	Final refinements (hardware, software, alerts).
Week 13 (Sep 29–Oct 2)	Write the final report and prepare for Milestone 2 submission.

11. The Team

- Project Lead – Josh Prithveraj
 - Oversee the entire planning process
 - Manage team timelines and task coordination.
 - Draft the Risk Management
 - Ensure clear team communication and documentation standards are in place.
 - Consolidate all work into the final written project plan for submission.
- Research Lead – Jessica Wiehe
 - Conduct background research on driver fatigue detection technologies and best practices.
 - Summarize findings in the plan to justify the need for your innovation.
 - Identify gaps in current solutions that your team's innovation will address.
- Hardware Design Lead – Hannes Smit
 - Develop preliminary design schematics for the glasses (sensor layout, components like IR, MPU6050, ESP32).
 - Define hardware needs, power sources, and system integration (e.g. Bluetooth/Wi-Fi connectivity).
 - Document component choices and justify them.
- Software Development Leads (2) – Mthokozisi Mahlangu & Itumeleng Ndlovu (Jessica Wiehe will join in M2)
 - Draft the software architecture, including how sensor data flows into the ML model.
 - Plan out how the glasses and dashboard system will communicate (e.g. mobile app, dashboard alerts).
 - Specify the tech stack and tools (e.g. TensorFlow, Arduino IDE).
- AI/ML Specialists – Kgotso Matemotja (Kaelo Molebatsi will assist when complete with User Experience Design)

- Design a plan for data collection (e.g. eye blinks, head movement).
- Propose the model structure (e.g. CNN for blink detection, LSTM for temporal fatigue detection).
- Detail how ML will work on edge devices for real-time processing.
- User Experience (UX) Designer – Kutenda Munatsi & Kaelo Molebatsi
 - Design dashboard mock-ups and user alert interfaces (visual and audio).
 - Ensure the alerts are user-friendly and ergonomically effective (sound/haptic/visual).
 - Ensure the design is inclusive, accessible, and intuitive.
- Testing and Quality Assurance Lead – Ayanda Cetshwayo
 - Develop a preliminary testing plan (how accuracy, latency, and reliability will be measured).
 - Define quality benchmarks and prepare the testing protocols.
 - Plan test environments (e.g. different lighting/driving conditions).

12. Required Hardware, Software and Tools

12.1. Hardware with prices

- ESP32 x2 (R120.00)
- IR Sensor TCRT5000 x2 (R56.00)
- BC547 Transistor x3 (R9.00)
- Coin Vibration Motor (3V) x2 (R75.00)
- 5V Active Buzzer x2 (R24.00)
- 4.7k Ω Resistor (1/4W) x3 (R3.00)
- 3.7V Li-Po Battery (1000mAh) x2 (R320)
- MPU6050 (Accelerometer/Gyroscope) x2 (R140.00)
- Raspberry Pi 4 Model B (4GB RAM) x1 (R1500 - R2000.00)
- Breadboard x1 (R50.00 - R100.00)
- Voltage regulator x2 (R20.00 - R40.00)
- TP4056 Li-Po Charging Module x2 (R30.00 - R50.00)
- Jumper Wires (Male–Male, Male–Female) – (R30)
- Flexible Hookup Wire (22–28 AWG) – (R100)

- Dupont Connector Kit (with crimp pins) – (R120)
- Heat Shrink Tubing Kit – (R60)
- Header Pin Strips (Male & Female) – (R25)
- JST Connectors – (R80)
- 220Ω resistors for IR LEDs (R10)
- Mini on/off switch (R40)
- Boost convertor (3.3V to 5V) (R30)
- USB-to-TTL adapter (R60–R100)
- Perfboard or Veroboard (R20–R40)
- Soldering Iron if it is not provided by the campus.
- Solder flux (R50)

12.2. AI and Machine Learning Tools

- Google Colab / Jupyter Notebooks
- Edge Impulse
- TensorFlow Lite

12.3. Connectivity and Integration

- Bluetooth / BLE
- Wi-Fi (ESP32)
- Firebase / Custom Web API

12.4. Mobile and Dashboard Development

- Flutter / Android Studio
- Flask / Node.js (on Raspberry Pi or Cloud)

12.5. Testing and Development Tools

- Serial monitor / Plotter (Arduino IDE)
- Multimeter / Oscilloscope (physical tools)

12.6. Design and Simulation Tools

- Tinker CAD

Project Design

1. Hardware Specifications

1.1 Core components

- ESP32 Microcontroller
- Raspberry Pi 4 Model B 4GB RAM
- MPU6050
- IR Sensor
- Transistor BC547
- Vibrator
- 5 Volt Buzzer
- 4.7K resistor
- 3.7V Battery
- Glasses Frame

1.2 Hardware Architecture for Anti-Sleep Glasses

The hardware architecture of the Anti-Sleep Glasses is organized into three layers: Sensing Layer, Processing and Control Layer, and Feedback and Communication Layer. These layers work cohesively to detect drowsiness through eye blink and head movement monitoring, process data in real-time, and deliver alerts to the user via visual, auditory, and haptic feedback. The architecture emphasizes modularity, power efficiency, and ergonomic integration into a lightweight glasses frame, ensuring comfort and reliability for long-distance drivers and other users.

1.2.1 Sensing Layer

This layer is responsible for capturing raw data related to eye blinks and head movements, which are critical indicators of drowsiness. It comprises sensors mounted on the glasses frame, designed for accurate and non-intrusive monitoring.

- **IR Sensor (TCRT5000):**
 - **Function:** Detects eye blinks by emitting infrared light and measuring reflections with a photodiode/phototransistor pair.
 - **Placement:** Embedded in a compartment on the bottom inner-left rim of the glasses frame, angled to align with the user's eye.
 - **Operation:** The IR LED emits light (controlled via GPIO 4 through a BC547 transistor), and the phototransistor generates an analog voltage proportional to reflected light, read by the ESP32's ADC (GPIO 34).

- **Specifications:** Operates at 3.3V, draws ~20mA (IR LED) and ~0.1mA (phototransistor), with a 220Ω resistor to limit LED current.
- **Ambient Light Protection:** A black plastic shield surrounds the sensor to minimize interference from ambient IR light, ensuring reliable detection in varied lighting conditions (5–1000 lux).
- **Notes:** The sensor's analog output is conditioned using an LM358 op-amp to amplify the signal for stable ADC readings. Pulsing the IR LED at a 10% duty cycle reduces power consumption to ~2mA.
- **MPU6050 (Accelerometer/Gyroscope):**
 - **Function:** Tracks head movements (e.g. tilting, nodding) to detect drowsiness patterns.
 - **Placement:** Mounted on the right temple near the ESP32 to capture head orientation accurately.
 - **Operation:** Communicates via I2C (SDA on GPIO 21, SCL on GPIO 22) with the ESP32, providing 6-axis data (3-axis accelerometer, 3-axis gyroscope).
 - **Specifications:** Operates at 3.3V, draws ~3.5mA in active mode, and supports a low-power mode (~0.01mA) for idle periods.
 - **Notes:** Built-in 4.7kΩ pull-up resistors on SDA/SCL lines ensure reliable I2C communication. The MPU6050's Digital Motion Processor (DMP) is utilized for pre-processed motion data, reducing ESP32 computational load.

1.2.2 Processing and Control Layer

This layer processes sensor data, executes drowsiness detection logic, and manages power distribution. It is centred around the ESP32 microcontroller, with the Raspberry Pi 4 providing supplementary processing for complex ML tasks.

- **ESP32-WROOM-32 Microcontroller:**
 - **Function:** Serves as the central controller, collecting sensor data, performing lightweight ML inference, and triggering alerts.
 - **Placement:** Housed in a 30x20x5mm compartment on the right temple, secured with vibration-damping adhesive.
 - **Operation:**

- Collects IR sensor data via ADC (GPIO 34) and MPU6050 data via I2C (GPIO 21/22).
 - Runs a lightweight CNN model (TensorFlow Lite) for edge-based blink detection when connectivity to the Raspberry Pi is unavailable.
 - Controls alert devices (LED, buzzer, vibration motor) via GPIO pins (4, 5, 18) through BC547 transistors.
 - Manages Bluetooth Low Energy (BLE) and Wi-Fi communication for data transmission to the Raspberry Pi or mobile app.
- **Specifications:** Dual core 240MHz CPU, 520KB SRAM, 3.3V logic, ~80mA idle, ~160mA active (Wi-Fi/BLE), ~10mA in deep sleep.
- **Power Optimization:** Utilizes deep sleep mode (~10mA) when idle, waking every 100ms for sensor polling, achieving a ~60mA average draw.
- **Notes:** GPIO pins are carefully assigned to avoid conflicts (e.g. GPIO 1/3 reserved for UART debugging). The ESP32's onboard regulator supplies 3.3V to sensors and alert devices.
- **Raspberry Pi 4 Model B (4GB RAM):** (take out)
 - **Function:** Acts as a central hub for advanced ML inference (CNN+LSTM models) and dashboard communication.
 - **Placement:** External to the glasses, typically mounted in the vehicle (e.g. near the dashboard).
 - **Operation:**
 - Receives pre-processed sensor data from the ESP32 via BLE as JSON packets (e.g. {"blink duration": 0.6, "head tilt": 15}).
 - Runs TensorFlow Lite models to compute drowsiness probability scores within 500ms.
 - Sends audio-visual alerts to the car's infotainment system via Bluetooth and logs data for mobile app syncing.
 - **Specifications:** Quad-core ARM Cortex-A72 @ 1.5GHz, 4GB RAM, supports USB, HDMI, and Ethernet for integration.
 - **Notes:** Operates in headless mode for efficiency, running Python scripts (pyserial) to process ESP32 data. Desktop notifications or audio beeps are optional for debugging.

- **Power Management:**

- **3.7V Li-Po Battery (1000mAh):** Powers the glasses, housed in a 40x30x5mm compartment on the left temple.
- **TP4056 Charging Module:** Manages battery charging via a micro-USB port, with overcharge/discharge protection. Charging time is ~1–2 hours at 500mA.
- **MT3608 Boost Converter:** Steps up 3.7V to 5V for the buzzer (~35mA, ~85dB), ensuring optimal performance. Efficiency is ~93%, minimizing power loss.
- **Mini On/Off Switch:** Controls power to the ESP32 and components, enhancing user control and safety.
- **Notes:** The ESP32 monitors battery voltage via an ADC pin, triggering a low-battery alert (alternating LED flashes) at <3.4V. Total optimized current draw is ~104.1mA, yielding ~9.6 hours runtime, meeting the 8+ hour requirement.

1.2.3 Feedback and Communication Layer

This layer delivers real-time alerts to the user and communicates with external systems (car dashboard, mobile app) to enhance safety and data logging.

- **Alert Mechanisms:**

- **High-Brightness LED:**
 - **Function:** Provides visual feedback, blinking red rapidly when drowsiness is detected, or static green for normal operation.
 - **Placement:** Mounted in a 5x5mm compartment on the top inner-right rim, positioned in the user's peripheral vision.
 - **Operation:** Controlled via ESP32 GPIO (e.g. GPIO 16), drawing ~2mA with a 1kΩ resistor.
 - **Notes:** Red/green colour coding ensures intuitive feedback. Brightness is optimized for visibility in daylight and night conditions.
- **5V Active Buzzer:**
 - **Function:** Emits audible alerts (~85dB) for noisy environments, activated in short beeps (5% duty cycle).
 - **Placement:** Positioned in the left temple near the ear, with an acoustic channel to direct sound.

- **Operation:** Controlled via GPIO 5 through a BC547 transistor, powered by the MT3608 boost converter for 5V operation.
 - **Notes:** The buzzer's low duty cycle reduces average current to ~1.25mA, preserving battery life.
- **3V Coin Vibration Motor:**
 - **Function:** Delivers haptic feedback for subtle alerts, ideal for quiet or shared spaces.
 - **Placement:** Embedded in the right temple, secured with vibration-damping adhesive for comfort.
 - **Operation:** Controlled via GPIO 18 through a BC547 transistor, drawing ~80mA when active (5% duty cycle, ~4mA average).
 - **Notes:** The motor's placement ensures feedback is felt without disrupting the user's focus.
- **Communication Interfaces:**
 - **Bluetooth Low Energy (BLE):**
 - **Function:** Primary method for ESP32-to-Raspberry Pi and ESP32-to-mobile app communication, transmitting drowsiness logs every 1–2 seconds.
 - **Operation:** Utilizes the ESP32's built-in Bluetooth stack, with a ~10m range and ~30mA average draw (25% duty cycle).
 - **Notes:** BLE ensures low power consumption and reliable in-car connectivity. Fallback to local ESP32 inference occurs if connectivity is lost.
 - **Wi-Fi (Optional):**
 - **Function:** Supports longer-range communication or cloud integration (e.g. Firebase for data logging).
 - **Operation:** Configured via ESP32 Wi-Fi libraries, connecting to a local network or hotspot.
 - **Notes:** Wi-Fi is reserved for non-critical tasks due to higher power consumption (~100mA active).
 - **Dashboard Integration:**

- **Function:** Sends audio-visual alerts to the car's infotainment system via the Raspberry Pi's Bluetooth stack.
 - **Operation:** The Raspberry Pi forwards alerts to the car's speakers, playing warning tones or messages.
 - **Notes:** Ensures seamless integration with modern vehicle systems, enhancing alert effectiveness.
- **Physical Integration:**
 - **Glasses Frame:** Made of lightweight ABS plastic (~30g), housing all components in ergonomic compartments:
 - Right Temple: ESP32, vibration motor, MPU6050.
 - Left Temple: Battery, TP4056, buzzer.
 - Bottom Inner-Left Rim: IR sensor.
 - Top Inner-Right Rim: LED.
 - **Weight Distribution:** Total weight is ~80g, balanced symmetrically to avoid strain.
 - **Ergonomic Features:** Adjustable nose pads, silicone temple tips, and ventilation slots for heat dissipation (~0.5W at 160mA).
 - **Environmental Durability:** Targets IP54 rating (dust-protected, splash-resistant) with sealed compartments and epoxy-coated PCB joints.
 - **Notes:** The frame's design prioritizes comfort for 8+ hour wear, validated through user testing (Section 8).

1.3 Hardware Workflow

The hardware operates in a continuous loop to support real-time drowsiness detection:

1. **Data Acquisition:** The IR sensor and MPU6050 collect eye blink and head movement data at 10Hz (every 100ms).
2. **Signal Conditioning:** The IR sensor's analog signal is amplified by an LM358 op-amp, and MPU6050 data is filtered via its DMP.
3. **Data Processing:** The ESP32 preprocesses sensor data (e.g. noise filtering, feature extraction) and runs lightweight ML inference or forwards data to the Raspberry Pi via BLE.
4. **Alert Activation:** If drowsiness is detected, the ESP32 triggers local alerts (LED, buzzer, vibration) and signals the Raspberry Pi for dashboard alerts.

5. **Power Management:** The TP4056 ensures safe battery charging, and the ESP32's deep sleep mode optimizes power usage.
6. **Data Logging:** The Raspberry Pi logs drowsiness events, syncing with the mobile app for post-trip analysis.

1.4 Error Handling and Fault Tolerance

- **Sensor Failure:** The ESP32 detects IR sensor or MPU6050 malfunctions (e.g. out-of-range readings) and triggers a warning LED pattern (slow blinking red).
- **Power Failure:** A low-battery alert is activated at <3.4V, and a backup battery option (second 1000mAh Li-Po) ensures continuity.
- **Component Overheating:** Ventilation slots and low-power modes prevent ESP32 overheating. The BC547 transistors protect GPIO pins from high currents.
- **Connectivity Loss:** The ESP32 falls back to local CNN inference and stores data in flash memory for later syncing.

1.5 Hardware Development Plan

The hardware development aligns with the project timeline (Weeks 1–13):

- **Week 1–2:** Order components (ESP32, TCRT5000, MPU6050, etc.) and verify datasheets. Assemble initial prototype on a breadboard.
- **Week 3:** Test sensor integration (IR sensor, MPU6050) with the ESP32 using TinkerCAD simulations and physical setups.
- **Week 4–5:** Solder components onto a perfboard and integrate into the glasses frame. Test power management (TP4056, MT3608).
- **Week 6–7:** Validate alert mechanisms (LED, buzzer, vibration) and BLE communication with the Raspberry Pi.
- **Week 8–9:** Conduct lab tests for power consumption (~104.1 mA) and runtime (~9.6 hours). Iterate frame design for ergonomics.
- **Week 10–11:** Perform field tests in a driving simulator, optimizing sensor placement and alert timing.
- **Week 12:** Finalize hardware assembly, fix defects, and verify IP54 durability.
- **Week 13:** Document hardware specifications and submit for Milestone 2.

1.6 Scalability and Future Enhancements

- **Modular Design:** Components are mounted on a detachable PCB, allowing upgrades (e.g. higher-capacity battery, advanced sensors).

- **Backup Power:** A secondary 1000mAh battery or USB power bank can extend runtime to ~19.2 hours.
- **Cloud Connectivity:** Wi-Fi integration with Firebase enables fleet-wide fatigue monitoring.
- **Durability Upgrades:** Future iterations may target IP65 (water-resistant) for heavy rain conditions.
- **Cost Reduction:** Bulk purchasing and local sourcing can lower costs below \$50 per unit.

1.7 Connectivity and Pin Assignments

The ESP32 is a versatile microcontroller with multiple GPIO pins, analog-to-digital converters (ADCs), and communication interfaces (I2C, UART, etc.). Below is the connectivity plan for each component, including the role of the BC547B transistor and the 4.7kΩ resistor.

1. IR LED + Photodiode (TCRT5000 Reflective Sensor)

- **Function:** Detects eye blinks by emitting IR light and measuring reflections with a photodiode/phototransistor.
- **Connection:**
 - **IR LED:**
 - Anode: Connected to ESP32 3.3V output (or via a current-limiting resistor, e.g. 220Ω, to limit current to ~20mA).
 - Cathode: Connected to a digital output pin (e.g. GPIO 4) via a BC547B transistor for switching.
 - **BC547B Transistor:**
 - Base: Connected to GPIO 4 through a 4.7kΩ resistor to limit base current.
 - Collector: Connected to IR LED cathode.
 - Emitter: Connected to GND.
 - Thermal consideration: BC547 handles low power well, but avoid sustained current near its max rating (~100mA). For the vibration motor (>70mA), ensure a good layout and short switching time.

- Flyback diodes: Add a 1N4007 diode across all inductive loads:
 1. Cathode: Power side of load
 2. Anode: Transistor collector
 - **Photodiode/Phototransistor:**
 - Collector: Connected to an analog input pin (e.g. GPIO 34, ADC1_CH6) with a pull-up resistor (e.g. 10kΩ to 3.3V).
 - Emitter: Connected to GND.
 - Output: The phototransistor generates a voltage proportional to reflected IR light, read by the ADC pin.
- **Pin Assignment:**
 - GPIO 4 (Digital Output): Controls IR LED via BC547B.
 - GPIO 34 (ADC1_CH6, Analog Input): Reads phototransistor output.
- **Positioning:**
 - Mount the IR sensor inside the glasses frame so it aligns with the user's eye to ensure stable detection.
- **Ambient Light Protection:**
 - If false triggering occurs, add a black plastic shield around the sensor to reduce ambient IR interference.
- **Notes:**
 - GPIO 34 is input-only and ADC-compatible, ideal for analog signals.
 - The BC547B ensures the ESP32's GPIO (limited to ~20mA) can safely switch the IR LED's higher current.
 - Test IR LED current with a 220Ω resistor to avoid exceeding 20mA (per TCRT5000 datasheet).

2. Raspberry Pi 4 Model B (4GB RAM)

- **Function:** Acts as a central monitoring system, receiving drowsiness alerts and logs the data. Can provide audio and visual alerts using speakers or screen
- **Connection:**
 - Bluetooth Serial: Establishes a connection between the ESP32 and Raspberry Pi using Bluetooth Serial Port Profile.
 - ESP32 sends data over Bluetooth Serial
 - Raspberry Pi runs a Python script with pyserial to receive the data.
- **Notes:**
 - Ensure the Raspberry Pi is paired and trusted with the ESP32.
 - Use headless mode or GUI depending on the project requirements
 - Optional: Add desktop notifications or audio beeps.

3. MPU6050 (Accelerometer/Gyroscope)

- **Function:** Tracks head movements to detect drowsiness patterns (e.g. head tilt).
- **Connection:**
 - Interface: I2C (SDA/SCL lines).
 - VCC: Connected to ESP32 3.3V output.
 - GND: Connected to ESP32 GND.
 - SDA: Connected to GPIO 21 (default I2C SDA pin).
 - SCL: Connected to GPIO 22 (default I2C SCL pin).
 - Optional: INT (interrupt pin) can be connected to GPIO 15 for motion detection interrupts (if needed).
- **Pin Assignment:**
 - GPIO 21 (SDA): I2C data line.
 - GPIO 22 (SCL): I2C clock line.
 - (Optional) GPIO 15: Interrupt pin (if used).
- **Notes:**

- Use the MPU6050's built-in Digital Motion Processor for more accurate head movement data if the library supports it.
- The MPU6050 operates at 3.3V, matching the ESP32's logic level.
- Ensure pull-up resistors (e.g. 4.7k Ω) are present on SDA/SCL lines (often built into the MPU6050 module).
- The I2C address is typically 0x68 (check datasheet).

4. Buzzer (5V Active Buzzer)

- **Function:** Provides audible alerts when drowsiness is detected.
- **Connection:**
 - Positive Pin: Connected to ESP32 3.3V or a 5V boost converter (if 5V is required).
 - Negative Pin: Connected to the collector of a BC547B transistor.
 - **BC547B Transistor:**
 - Base: Connected to GPIO 5 through a 4.7k Ω resistor.
 - Collector: Connected to buzzer negative pin.
 - Emitter: Connected to GND.
- **Pin Assignment:**
 - GPIO 5 (Digital Output): Controls buzzer via BC547B.
- **Notes:**
 - If the buzzer operates reliably at 3.3V (~85dB per datasheet), connect directly to 3.3V; otherwise, use a 3.7V-to-5V boost converter.
 - The BC547B handles the buzzer's current (~35mA), protecting the ESP32 GPIO.
 - Test buzzer activation with a simple PWM signal if variable tones are desired.
 - Use MT3608 or similar 3.7V to 5V boost module.
 - Connect boost output to buzzer V+.
 - GND shared with ESP32 and TP4056.

5. Vibration Motor (3V Coin Type)

- **Function:** Provides haptic feedback for drowsiness alerts.
- **Connection:**
 - Positive Pin: Connected to ESP32 3.3V (or battery 3.7V if within motor tolerance).
 - Negative Pin: Connected to the collector of a BC547B transistor.
 - **BC547B Transistor:**
 - Base: Connected to GPIO 18 through a 4.7k Ω resistor.
 - Collector: Connected to vibration motor negative pin.
 - Emitter: Connected to GND.
- **Pin Assignment:**
 - GPIO 18 (Digital Output): Controls vibration motor via BC547B.
- **Notes:**
 - The vibration motor (~80mA) exceeds ESP32 GPIO current limits, requiring the BC547B for switching.
 - Ensure the motor's voltage tolerance (2.5–3.7V per datasheet) matches the 3.3V supply or adjust with a regulator.
 - Place in the temple of the glasses frame. Use hot glue gun or vibration damping adhesive to secure it and ensure the vibration is felt without being disruptive.
 - Ensure it doesn't exceed 80mA. Power from battery directly if needed, via transistor switching.

6. Bluetooth/Wi-Fi

- **Function:** Enables communication with the dashboard system (Raspberry Pi) for audio-visual alerts.
- **Connection:**
 - Built into the ESP32-WROOM-32 module; no additional pins required.
 - Configuration: Handled via software (e.g. ESP32 Bluetooth Serial or Wi-Fi libraries).

- **Pin Assignment:** None (internal to ESP32).
- **Notes:**
 - Reserve sufficient program memory for Bluetooth/Wi-Fi stack.
 - Test connectivity range (~10m for Bluetooth) to ensure dashboard integration.

B. Power and Debugging Pins

- **Power Pins:**
 - **3.3V Output:** Supplied by ESP32's onboard regulator (from 3.7V Li-Po battery). Used for IR sensor, MPU6050, buzzer, and vibration motor.
 - **GND:** Multiple GND pins (e.g. GND1, GND2) for component grounding.
 - **VIN/5V:** If a 5V boost converter is used for the buzzer, connect the battery to a boost module, then to ESP32 VIN.
 - **Battery Management:**
 - **3.7V Li-Po Battery (1000mAh):** Connected to ESP32 via a TP4056 charging/protection module.
 - **TP4056:**
 - Input: USB for charging (5V).
 - Output: 3.7V to ESP32 and components.
 - Connect battery +/- to TP4056 BAT+/- terminals.
 - **Notes:** Ensure the TP4056 includes overcharge/discharge protection (per datasheet).
- **Debugging Pins:**
 - **Serial UART:**
 - GPIO 1 (TX): Serial transmit for debugging output.
 - GPIO 3 (RX): Serial receive for programming.
 - **Notes:**

- Reserve GPIO 1 and 3 for USB-to-Serial communication during development.
- Use a USB-to-TTL adapter (e.g. FTDI module) for programming and debugging.

Component	ESP32 Pin	Type	Notes
IR Sensor (Photodiode)	GPIO 34	Analog Input	Reads conditioned signal from LM358 op-amp.
BC547 Base (via 4.7kΩ)	GPIO 4	Digital Output	Controls IR LED via transistor.
BC547 Base (Buzzer)	GPIO 5	Digital Output	Controls buzzer via transistor.
BC547 Base (Vibration motor)	GPIO 18	Digital Output	Controls vibration motor via transistor.
MPU6050 SDA	GPIO 21	I2C SDA	I2C data line; add 4.7kΩ pull-up to 3.3V.
MPU6050 SCL	GPIO 22	I2C SCL	I2C clock line; add 4.7kΩ pull-up to 3.3V.
Power (3.7V Li-Po)	VIN	Power Input	Via TP4056; use 3.3V Pro Mini for direct compatibility.
GND	GND	Ground	Multiple GND pins for components.

1.8 Calculate Total Power Consumption

To calculate the total power consumption, we need to estimate the current draw for each component in the system:

1. Component Current Draw Estimates

- **ESP32-WROOM-32: ~80mA (idle), ~160mA (Wi-Fi/Bluetooth active).:**
 - **Active Mode:** ~80- 160mA
 - **Sleep Mode:** ~10mA (using low-power modes like deep sleep, achievable with Arduino libraries).
 - **Assumption:** Assume active mode for worst-case calculation, as the Pro Mini will frequently process sensor data.
 - **Current Draw:** 10-20mA on low; 160-250mA on active
 - **Source:** (Last Minute Engineers, n.d.)
- **IR Sensor (TCRT5000):**
 - **IR LED:** ~20mA (with a 100Ω current-limiting resistor at 3.3V, per datasheet).
 - **Photodiode/Phototransistor:** ~0.1mA (negligible, as it's a passive sensor).
 - **LM358 Op-Amp** (from the provided diagram): ~0.7mA (typical quiescent current at 3.3V, per datasheet).
 - **Assumption:** IR LED is on continuously for eye blink detection. Can be reduced by pulsing (e.g. 10% duty cycle).
 - **Total Current Draw:** 20mA (IR LED) + 0.1mA (photodiode) + 0.7mA (LM358) = **20.8mA**.
 - **Source:** Vishay Semiconductors, 2025. *TCRT5000 Datasheet*; Texas Instruments, 2025. *LM358 Datasheet* (generic reference for LM358).
- **MPU6050 (Accelerometer/Gyroscope):**
 - **Active Mode:** ~3.5mA (at 3.3V, per datasheet).
 - **Sleep Mode:** ~0.01mA (possible with low-power settings).
 - **Assumption:** Active mode for continuous head movement detection.
 - **Current Draw:** 3.5mA.

- **Source:** InvenSense, 2025. *MPU6050 Datasheet* (generic reference, as MPU6050 is widely documented).
- **5V Buzzer:**
 - **Active:** ~35mA (at 5V, per datasheet). At 3.7V, current scales down to ~25mA (assuming linear voltage-current relationship).
 - **Assumption:** Buzzer is active 5% of the time (e.g. short beeps when drowsiness is detected).
 - **Average Current Draw:** $25\text{mA} \times 0.05 = 1.25\text{mA}$.
 - **Source:** CUI Devices, 2025. *CEM-1203(42) Buzzer Datasheet*.
- **Vibration Motor (3V Coin Type):**
 - **Active:** ~80mA (at 3.3V, per datasheet).
 - **Assumption:** Active 5% of the time (same as buzzer).
 - **Average Current Draw:** $80\text{mA} \times 0.05 = 4\text{mA}$.
 - **Source:** Adafruit Industries, 2025. *Vibration Motor Datasheet*.
- **BC547B Transistor and 4.7kΩ Resistor:**
 - **Transistor:** Negligible current draw when off; when on, it conducts buzzer/vibration motor current (accounted above).
 - **Base Current:** Through 4.7kΩ resistor from Pro Mini GPIO (3.3V).
 - Base current = $(3.3\text{V} - 0.7\text{V}) / 4.7\text{k}\Omega = 2.6\text{V} / 4.7\text{k}\Omega = \sim 0.55\text{mA}$.
 - **Current Draw:** 0.55mA.
 - **Source:** ON Semiconductor, 2025. *BC547 NPN Transistor Datasheet*.
- **Additional Components (from Diagram):**
 - **Power LED:** ~2mA (with a 1kΩ resistor at 3.3V).
 - **Push Button:** Negligible (only draws current when pressed, assume 0mA).

- **Total: 2mA.**

2. **Total Current Draw (Optimized)**

Summing the average current draws:

- ESP32 (deep sleep, 50% active): ~60mA
- IR Sensor (10% duty cycle): ~2.8mA
- MPU6050: ~3.5mA
- BLE (25% duty cycle): ~30mA
- Buzzer (5% duty cycle): ~1.25mA
- Vibration Motor (5% duty cycle): ~4mA
- BC547 + 4.7kΩ: ~0.55mA
- Power LED: ~2mA
- Total: ~104.1mA
- Runtime: 1000mAh / 104.1mA = 9.6 hours

Total Current Draw (Worst Case): ~152.1mA

Runtime (Unoptimized, 1000mAh): 1000mAh / 152.1mA = 6.58 hours

Total Current Draw (Optimized): ~104.1mA

Runtime (Optimized, 1000mAh): 1000mAh / 104.1mA = 9.6 hours

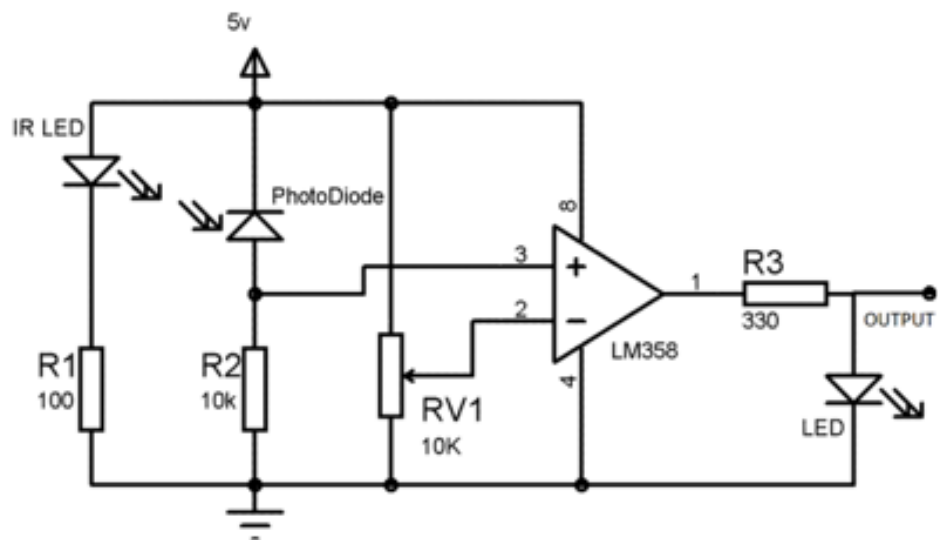
1.9 Power Optimization Strategy

To achieve the 8+ hour runtime with the ESP32's higher power, the following optimizations are implemented:

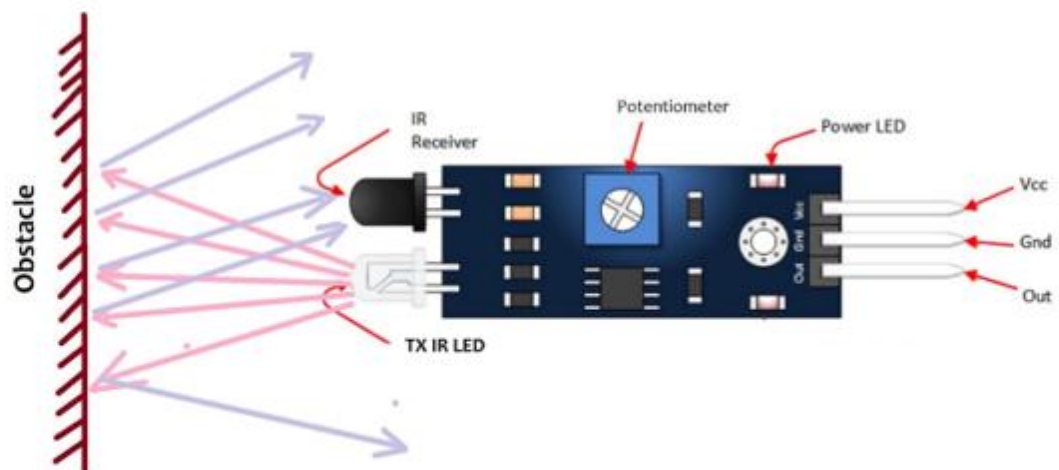
- **ESP32 Deep Sleep:** Reduces power to ~10mA when idle, waking every 100ms for sensor polling.
- **IR LED Pulsing:** Operates at 10% duty cycle, reducing current from 20mA to ~2mA.
- **Reduced BLE Transmission:** Sends data every 2 seconds (~25% duty cycle), lowering BLE power to ~30mA.

Result: Total current ~104.1mA, runtime ~9.6 hours, meeting the requirement.

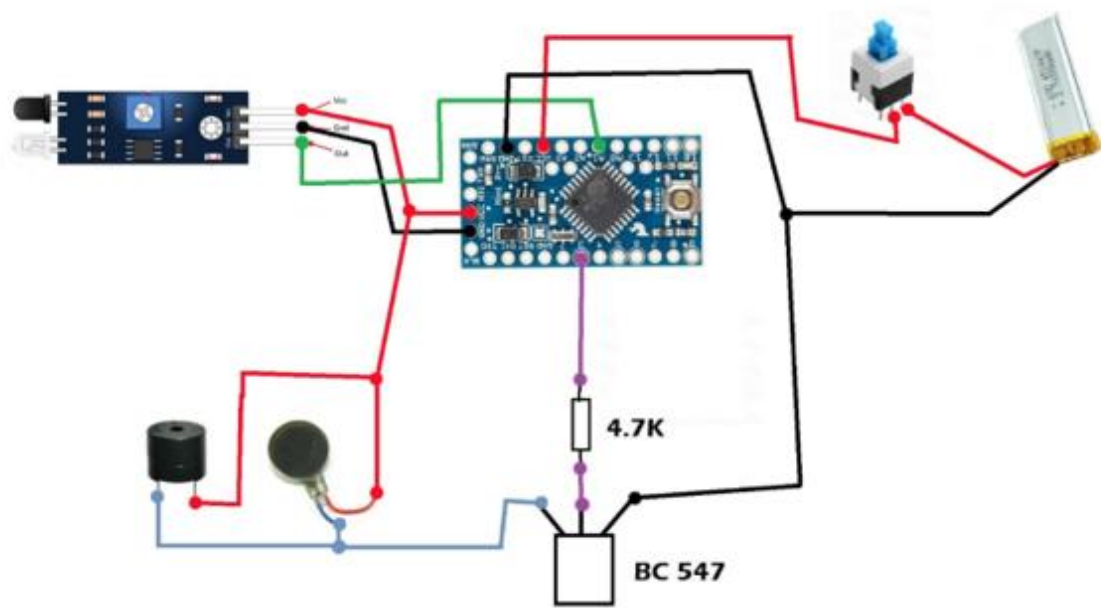
1.10 Circuit Diagram



1.11 How it works



1.12 Diagram



2. Software Design

2.1 Software Architecture

The software system is divided into three layers: Data Acquisition, Data Processing and ML Inference, and Alert and Communication. These layers ensure that sensor data is collected, processed, and acted upon within a 2-second latency for real-time drowsiness detection. The architecture is designed to be modular, allowing for easy updates, scalability, and adaptability.

2.1.1. Data Acquisition Layer

This layer is responsible for collecting raw sensor data from the IR sensor and MPU6050. It runs on the ESP32 microcontroller, programmed using the Arduino IDE in C/C++.

- **IR Sensor Data:** The IR sensor detects eye blinks by measuring reflected infrared light. The analog signal from the phototransistor is read via the ESP32's Analog-to-Digital-Convertor and converted into digital values representing blink duration and frequency.
- **MPU6050 Data:** The MPU6050 captures head movement through its 3-axis accelerometer and 3-axis gyroscope. Data is read via the I2C interface and processed to detect patterns such as head tilting or nodding, indicative of drowsiness.
- **Data Preprocessing:** The ESP32 applies basic preprocessing to reduce computational load on the ML model. A sliding window aggregates sensor data to capture temporal patterns.

2.1.2. Data Processing and ML Inference Layer

This layer processes sensor data and performs drowsiness detection using AI/ML models. It primarily runs on the Raspberry Pi 4, which has sufficient computational power for ML inference, though lightweight models are deployed on the ESP32 for edge processing.

- **ML Model Structure:**
 - **Convolutional Neural Network (CNN):** Used for classifying blink patterns based on IR sensor data. The CNN processes time-series data (blink duration, frequency, and eye aspect ratio) to identify prolonged blinks associated with drowsiness.
 - **Long Short-Term Memory (LSTM):** A temporal model that analyses sequences of blink and head movement data to detect fatigue progression. The LSTM takes input from the

CNN and MPU6050 data, learning patterns such as gradual increases in blink duration or irregular head movements.

- **Model Optimization:** The CNN and LSTM models are optimized for edge devices using TensorFlow Lite, reducing model size and inference time to meet the latency requirement. Quantization techniques are applied to ensure compatibility with the ESP32 and Raspberry Pi.
- **Data Flow:**
 - a. The ESP32 collects and preprocesses IR and MPU6050 data.
 - b. Pre-processed data is transmitted to the Raspberry Pi via Bluetooth Low Energy or Wi-Fi as structured packets.
 - c. The Raspberry Pi runs the CNN+LSTM model using TensorFlow Lite, producing a drowsiness probability score.
 - d. If the score exceeds a threshold, a drowsiness alert is triggered.
- **Edge Processing:** For scenarios with limited connectivity, a lightweight CNN model runs directly on the ESP32 to perform basic blink detection, reducing dependency on the Raspberry Pi. The ESP32 then triggers alerts if drowsiness is detected.

2.1.3. Alert and Communication Layer

This layer handles real-time alerts and communication with external systems (car dashboard and mobile app). It ensures that alerts are delivered promptly, and that data is logged for analysis.

- **Alert Mechanisms:**
 - **LED Alerts:** A high-brightness red LED (positioned on the glasses' upper frame) blinks rapidly when drowsiness is detected. A static green LED indicates normal operation, or the LED will only turn on when drowsiness is detected.
 - **Buzzer Alerts:** A 5V active buzzer emits short, repeating beeps to alert the user in noisy environments.
 - **Vibration Alerts:** A 3V coin vibration motor provides haptic feedback, embedded in the temple of the glasses for subtle yet effective alerts.
 - **Dashboard Alerts:** The Raspberry Pi sends audio-visual alerts to the car's infotainment system via Bluetooth, using the car's speakers to play a warning tone or message.
- **Communication Protocols:**

- **Bluetooth Low Energy (BLE):** Primary communication method between the ESP32 and Raspberry Pi. BLE ensures low power consumption and a range of 10meters, suitable for in-car connectivity. Data packets are sent every 1–2 seconds to maintain real-time updates.
- **Wi-Fi:** Used as a fallback for longer-range communication or when integrating with cloud-based services. The ESP32's Wi-Fi module connects to a local network or hotspot.
- **Mobile App Integration:** The Raspberry Pi forwards drowsiness logs to a mobile app via BLE or Wi-Fi. The app stores data in a local SQLite database or cloud-based Firebase for historical analysis.
- **Data Logging:** The Raspberry Pi logs drowsiness events in a JSON format, which is accessible via the mobile app or web dashboard for post-trip analysis.

2.2. Software Workflow

The software operates in a continuous loop to ensure real-time performance:

1. **Sensor Data Collection:** The ESP32 polls the IR sensor and MPU6050 at a frequency of 10Hz (every 100ms) to capture blink and head movement data.
2. **Preprocessing:** The ESP32 filters noise (e.g. using a low-pass filter for MPU6050 data) and computes features like blink duration and head tilt angle.
3. **Data Transmission:** Pre-processed data is sent to the Raspberry Pi via BLE as JSON packets.
4. **ML Inference:** The Raspberry Pi runs the CNN+LSTM model, producing a drowsiness score within 500ms.
5. **Alert Triggering:** If drowsiness is detected, the Raspberry Pi signals the ESP32 to activate local alerts (LED, buzzer, vibration) and sends dashboard alerts via Bluetooth.
6. **Data Logging:** Drowsiness events are logged on the Raspberry Pi and synced to the mobile app for visualization.

2.3. Error Handling and Fault Tolerance

- **Sensor Failure:** If the IR sensor or MPU6050 fails, the ESP32 logs an error and triggers a warning LED pattern (e.g. slow blinking red) to alert the user.

- **Connectivity Loss:** If BLE/Wi-Fi connectivity is lost, the ESP32 falls back to local ML inference (lightweight CNN) and stores data in its internal flash for later syncing.
- **Low Battery:** The ESP32 monitors battery voltage via an ADC pin and triggers a low-battery alert when voltage drops below 3.4V.
- **Model Misclassification:** The ML model uses conservative thresholds to minimize false positives, with periodic retraining to improve accuracy.

2.4 Tech Stack and Tools

The software leverages a combination of embedded, ML, and UI development tools to ensure efficiency and accessibility:

- **Microcontroller Programming:**
 - **Arduino IDE (C/C++):** Used for programming the ESP32 to handle sensor data collection, preprocessing, and alert triggering. Libraries like Wire.h (for I2C) and BluetoothSerial.h (for BLE) are utilized.
 - **MicroPython (Optional):** An alternative for rapid prototyping on the ESP32, particularly for testing communication protocols.
- **Machine Learning Framework:**
 - **TensorFlow Lite:** Deployed on the Raspberry Pi and ESP32 for lightweight ML inference. Supports quantized models for low-power devices.
 - **Edge Impulse:** Used for data collection, model training, and optimization. Simplifies the process of creating and deploying ML models for embedded devices.
 - **Google Colab / Jupyter Notebooks:** Used for initial model training and evaluation, leveraging Python libraries like TensorFlow, NumPy, and scikit-learn.
- **Communication Protocols:**
 - **Bluetooth Low Energy (BLE):** Implemented using the ESP32's built-in Bluetooth stack for low-power, reliable communication.
 - **Wi-Fi:** Used for optional cloud integration or longer-range communication, leveraging the ESP32's Wi-Fi capabilities.

- **Firebase / Custom Web API:** For cloud-based data storage and retrieval, enabling historical analysis via the mobile app or web dashboard.
- **Mobile and Dashboard Development:**
 - **Flutter:** Used to develop a cross-platform mobile app for Android and iOS. The app visualizes drowsiness logs, displays real-time alerts, and supports user configuration (e.g. alert preferences).
 - **Flask / Node.js:** Runs on the Raspberry Pi to serve a web-based dashboard, accessible via a browser or car infotainment system. Uses Chart.js for data visualization and Bootstrap for responsive design.
 - **Chart.js:** For rendering dynamic charts (e.g. drowsiness frequency over time) on the mobile app and web dashboard. Supports real-time updates and customizable visualizations.
 - **D3.js (Optional):** Used for advanced, custom visualizations if Chart.js lacks sufficient flexibility for specific use cases (e.g. interactive heatmaps of fatigue patterns).
- **Testing and Development Tools:**
 - **Arduino Serial Monitor / Plotter:** For debugging sensor data and communication logs.
 - **Wireshark:** For analysing BLE/Wi-Fi packet transmission and latency.
 - **Multimeter / Oscilloscope:** For validating hardware-software integration (e.g. checking signal timing).
- **Design and Simulation Tools:**
 - **TinkerCAD:** For simulating circuit behaviour and testing sensor integration before physical assembly.

2.5. Software Development Plan

The software development process is aligned with the project timeline (Weeks 1–13) and follows an iterative approach:

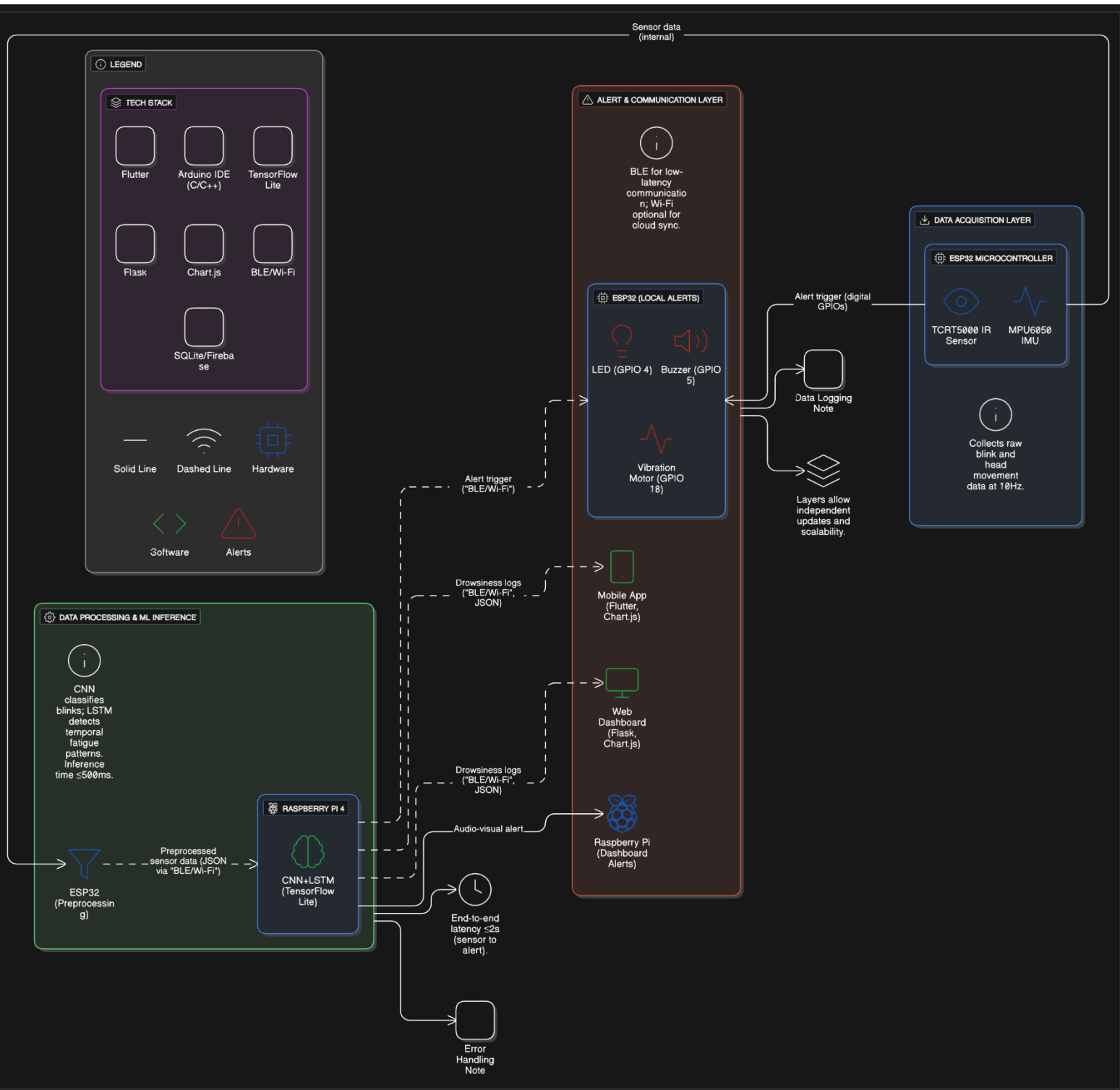
- **Week 1–2:** Set up the ESP32 development environment (Arduino IDE) and test basic sensor data collection (IR sensor, MPU6050).

- **Week 3:** Collect initial datasets for ML training (blink and head movement data) using video feeds and Edge Impulse.
- **Week 4:** Train the CNN+LSTM model in Google Colab, optimize for TensorFlow Lite, and deploy to the Raspberry Pi.
- **Week 5–6:** Implement BLE communication between the ESP32 and Raspberry Pi. Develop a basic Flutter app prototype for data logging.
- **Week 7–8:** Integrate the ML model with the ESP32 for edge processing. Test alert mechanisms (LED, buzzer, vibration) and dashboard integration.
- **Week 9:** Develop the web dashboard using Flask and Chart.js. Finalize the mobile app UI with real-time data visualization.
- **Week 10–11:** Conduct lab and field tests to validate latency, accuracy, and reliability. Optimize code based on test results.
- **Week 12:** Finalize software integration, fix bugs, and document the codebase.
- **Week 13:** Prepare the final software documentation and submit for Milestone 2.

2.6 Scalability and Future Enhancements

- **Cloud Integration:** Integrate with Firebase for long-term data storage and analysis, enabling fleet management systems to monitor driver fatigue across multiple vehicles.
- **OTA Updates:** Implement over-the-air firmware updates for the ESP32 to support model retraining and feature additions.
- **Adaptive Thresholds:** Develop algorithms to dynamically adjust drowsiness thresholds based on user-specific patterns (e.g. baseline blink frequency).
- **Multi-Environment Support:** Extend the system for non-driving use cases (e.g. machine operators, night-shift workers) by customizing alert patterns and data logging.

2.7. Software flow



3. User Experience and User Design

3.1. Introduction

The user experience and user interface design are critical to ensuring the system is intuitive, effective, and comfortable for long-term use, particularly for long-distance drivers, transportation professionals, and shift workers. This section outlines the UX objectives, alert system design, UI components, and testing plan to deliver a seamless and inclusive experience that promotes safety and user engagement.

3.2. Objectives

The UX and UI design aims to achieve the following:

- **Enhance Safety:** Deliver clear, immediate alerts to prevent micro-sleeps and fatigue-related accidents, with a target of ≤ 2 -second alert comprehension time.
- **Ensure Comfort and Wearability:** Design an ergonomic glasses frame and non-intrusive alerts for 8+ hours of continuous use, validated by $\geq 80\%$ user satisfaction in testing.
- **Provide Actionable Insights:** Offer a mobile app and dashboard with data visualizations to help users understand and manage fatigue patterns.
- **Promote Accessibility:** Support diverse users, including those with hearing or visual impairments, through customizable alerts and inclusive UI design.
- **Enable Seamless Integration:** Facilitate connectivity with car infotainment systems and mobile devices for real-time alerts and data logging.
- **Support Adaptability:** Allow customization of alert preferences and extend usability to non-driving contexts.

3.3. User Personas and Use Cases

To ground the UX design, the following user personas and use cases are considered:

- **Persona 1: Long-Distance Truck Driver (Male, 40, Works Night Shifts):**
 - **Needs:** Reliable drowsiness alerts during 10-hour drives, minimal distraction, integration with truck's audio system.

- **Use Case:** Receives a vibration alert during a late-night drive, followed by an audio warning via the car's speakers, prompting a rest stop.

- **Persona 2: University Student (Female, 22, Drives Late After Study Sessions):**
 - **Needs:** Lightweight glasses, affordable, mobile app for tracking fatigue trends.
 - **Use Case:** Notices a red LED blinking during a drive home, checks the mobile app post-trip to review drowsiness events, and adjusts sleep habits.

- **Persona 3: Night-Shift Factory Worker (Male, 35, Operates Heavy Machinery):**
 - **Needs:** Non-intrusive alerts, haptic feedback for noisy environments, durability in industrial settings.
 - **Use Case:** Feels a vibration alert while operating machinery, takes a break, and reviews fatigue logs to discuss shift adjustments with the employer.

3.4. Alert System Interface Design

The alert system is designed to deliver immediate feedback when drowsiness is detected, using a combination of visual, auditory, and haptic modes. Each alert is optimized for specific environmental and user needs, ensuring effectiveness without overwhelming the user. The table below summarizes the alert mechanisms:

Modality	Component	Placement	Purpose	User Impact
Visual	High-Brightness LED	Top inner-right rim (peripheral vision)	Signals system status and drowsiness alerts	Non-intrusive, visible in day/night conditions
Auditory	5V Active Buzzer (~85dB)	Left temple, near ear with acoustic channel	Provides audible warnings in noisy environments	Attention-grabbing, minimizes auditory stress
Haptic	3V Coin Vibration Motor	Right temple, vibration-damped	Delivers subtle alerts for quiet or shared spaces	Ideal for hearing-impaired users or low-visibility conditions

3.4.1 Visual Alerts (LED)

- **Function:** The LED provides system status and drowsiness alerts with distinct modes:
 - **Static Green:** System active, no drowsiness detected.
 - **Rapid Blinking Red (2Hz):** Drowsiness detected, immediate action required (e.g. take a break).
- **Operation:** Controlled via ESP32 GPIO 16 with a 1kΩ resistor, drawing ~2mA. Brightness is optimized for 5–1000 lux conditions.
- **Accessibility:** For colourblind users, the LED's blinking pattern is the primary cue, avoiding reliance on colour differentiation.
- **User Interaction:** The LED is positioned in the peripheral vision to avoid distracting the user's focus on the road. Testing will ensure ≥90% of users notice the alert within 2 seconds.

3.4.2. Auditory Alerts (Buzzer)

- **Function:** Emits short, repeating beeps (~85dB, 5% duty cycle) when drowsiness is detected, effective in high-noise environments.
- **Operation:** Controlled via GPIO 5 through a BC547 transistor, powered by an MT3608 boost converter for 5V operation, averaging ~1.25mA.
- **Sound Design:** Uses a 500ms beep with 2-second intervals to avoid auditory fatigue. The acoustic channel directs sound to the ear, minimizing leakage.

- **Accessibility:** Volume is adjustable via firmware to accommodate users with hearing sensitivities. Haptic feedback serves as a fallback for hearing-impaired users.
- **User Impact:** Designed to prompt immediate awareness without startling, validated by user feedback targeting $\leq 5\%$ annoyance rate.

3.4.3. Haptic Alerts (Vibration Motor)

- **Function:** Provides subtle haptic feedback for environments where visual or auditory alerts are less effective.
- **Operation:** Controlled via GPIO 18 through a BC547 transistor, drawing $\sim 4\text{mA}$ at 5% duty cycle. Secured with vibration-damping adhesive for comfort.
- **Placement:** Embedded in the right temple to ensure the vibration is felt near the user's head without disrupting focus.
- **Accessibility:** Critical for hearing-impaired users and effective in low-visibility conditions. Customizable intensity levels will be tested in Milestone 2.
- **User Impact:** Targets $\geq 85\%$ user detection rate within 2 seconds, ensuring discreet yet effective feedback.

3.5. Alert Workflow

1. **Drowsiness Detection:** The ML model (CNN+LSTM) identifies a drowsiness score > 0.7 , triggering the ESP32.
2. **Primary Alert:** The LED blinks red, and the vibration motor activates simultaneously to ensure immediate notice.
3. **Secondary Alert:** If no user response (e.g. head movement) is detected within 5 seconds, the buzzer activates to escalate urgency.
4. **Dashboard Integration:** The Raspberry Pi sends an audio alert to the car's infotainment system via Bluetooth, playing a warning tone or message (e.g. "Drowsiness detected, please take a break").
5. **User Response:** The system logs whether the user takes action (e.g. stops driving) for post-trip analysis via the mobile app.

3.6. User Interface Design

The UI bridges the physical hardware with the user's understanding of system status, alert events, and fatigue trends. It comprises a mobile app and a web dashboard, accessible via smartphones, tablets, or car

infotainment systems. The UI is designed to be intuitive, responsive, and inclusive, supporting real-time monitoring and historical analysis.

3.7. Purpose and Functionality

- **Real-Time Monitoring:** Displays live system status (e.g. active, drowsy) and alerts, ensuring drivers stay informed without distraction.
- **Historical Analysis:** Visualizes fatigue patterns (e.g. blink frequency, drowsiness events) to help users identify trends and adjust habits.
- **Customization:** Allows users to configure alert preferences (e.g. disable buzzer, adjust vibration intensity) and UI themes (light/dark mode).
- **Data Logging:** Stores drowsiness events in a local SQLite database (mobile app) or Firebase (cloud) for post-trip review.
- **Car Integration:** Forwards alerts to the car's infotainment system via Bluetooth, enhancing alert effectiveness in vehicles.

3.8. Mobile App Interface

- **Platform:** Cross-platform (Android/iOS) developed using Flutter for rapid deployment and consistent UX.
- **Key Features:**
 - **Home Screen:** Displays real-time system status (e.g. "System Active: No Drowsiness") with a color-coded indicator (green/yellow/red).
 - **Live Data Panel:** Shows dynamic charts (via Chart.js) for blink frequency, head tilt angles, and drowsiness probability scores, updated every 2 seconds.
 - **Alert History:** A scrollable timeline of drowsiness events with timestamps, severity (low/medium/high), and user response (e.g. break taken).
 - **Settings:** Configurable options for alert modalities (LED/buzzer/vibration), notification preferences, and theme (light/dark mode for day/night use).
- **Accessibility:**
 - High-contrast UI for visually impaired users.
 - Voice-over support for screen readers.

- Touch-friendly buttons ($\geq 44\text{px}$) for ease of use on mobile devices.
- **Data Flow:**
 - The ESP32 sends pre-processed sensor data to the Raspberry Pi via BLE.
 - The Raspberry Pi forwards drowsiness logs to the mobile app via BLE or Wi-Fi.
 - Data is stored locally (SQLite) or synced to Firebase for cloud access.

3.9. Web Dashboard Interface

- **Platform:** Hosted on the Raspberry Pi using Flask, accessible via a browser or car infotainment system.
- **Key Features:**
 - **Overview Panel:** Summarizes recent drowsiness events, system uptime, and battery status.
 - **Analytics Panel:** Displays advanced visualizations, including:
 - Line graphs for blink frequency and head movement over time.
 - Heatmaps for fatigue patterns (e.g. peak drowsiness hours).
 - Bar charts for alert frequency by severity.
 - **Alert Log:** A searchable table of events with filters for date, severity, and user response.
- **Chart.js Integration:**
 - Renders dynamic, real-time charts with minimal latency.
 - Supports data decimation for large datasets (e.g. 24-hour fatigue logs).
 - Customizable colours and annotations (e.g. threshold lines for drowsiness scores).
- **Accessibility:**
 - Responsive design for various screen sizes (mobile, tablet, car display).
 - Keyboard navigation for users with motor impairments.

- ARIA labels for screen reader compatibility.
- **Data Flow:**
 - The Raspberry Pi processes ML inference and logs data to a local JSON file.
 - Flask serves the dashboard, querying the JSON file or Firebase for real-time updates.

3.10. Car Infotainment Integration

- **Function:** Delivers audio-visual alerts to the car's infotainment system for enhanced safety.
- **Operation:**
 - The Raspberry Pi connects to the car's Bluetooth stack, sending warning tones or voice messages (e.g. "Drowsiness detected").
 - Visual alerts (e.g. red warning icon) are displayed on the car's screen if supported.
- **User Impact:** Ensures alerts are noticeable in high-distraction environments (e.g. heavy traffic), targeting $\geq 90\%$ driver response rate.
- **Testing:** Integration will be validated with a driving simulator and real-world vehicles in Weeks 10–11.

3.11. Accessibility and Inclusivity

The UX design prioritizes inclusivity to support diverse users:

- **Hearing-Impaired Users:** Haptic feedback and visual alerts ensure functionality without auditory cues.
- **Visually Impaired or Colourblind Users:** High-contrast UI, blinking patterns (vs. colour reliance), and voice-over support.
- **Motor Impairments:** Touch-friendly buttons and keyboard navigation for the dashboard.
- **Non-Native Speakers:** Simple, icon-based UI with multilingual support (planned for Milestone 2).
- **Cultural Considerations:** Neutral colour schemes and universal symbols to avoid cultural misinterpretation.

3.12. Development Plan

The UX and UI development aligns with the project timeline:

- **Week 5–6:** Design initial wireframes (Figma) and prototype the Flutter mobile app.
- **Week 7:** Develop the Flask-based web dashboard with Chart.js visualizations.
- **Week 8:** Conduct lab testing for alert comprehension and UI usability.
- **Week 9:** Integrate the mobile app and dashboard with the Raspberry Pi via BLE/Wi-Fi.
- **Week 10–11:** Perform field testing, collect user feedback, and iterate designs.
- **Week 12:** Finalize UI components, fix usability issues, and document the UX design.

3.13. Future Enhancements

- **Personalized Alerts:** Use ML to adapt alert thresholds based on user-specific fatigue patterns (e.g. baseline blink frequency).
- **Augmented Reality (AR) Integration:** Display alerts on AR glasses for enhanced visibility (long-term goal).
- **Fleet Management Support:** Integrate with fleet management systems to monitor driver fatigue across multiple vehicles.
- **Voice Control:** Allow users to interact with the mobile app via voice commands for hands-free operation.

3.14. Summary

The UX and UI design transforms the Anti-Sleep Glasses into a smart, user-adaptive platform that goes beyond reactive alerts. By combining intuitive alert mechanisms, real-time monitoring, and actionable data visualizations, the system empowers users to stay safe and manage fatigue effectively. The design's focus on accessibility, comfort, and seamless integration ensures broad adoption and long-term impact, validated through rigorous user testing and iterative refinement.

Chart.js Latest (4.4.9) Home API Samples Ecosystem

Bar chart provides a way of showing data values represented as vertical bars. It is sometimes used to show trend data, and the comparison of multiple data sets side by side.

Month	Value
January	65
February	58
March	80
April	81
May	56
June	54
July	40

[illegible]

4. AI/ML model for anti-drowsiness

4.1. Introduction

The Anti-Sleep Glasses system relies on artificial intelligence and machine learning to detect driver drowsiness in real-time, using eye blink and head movement data from the TCRT5000 and MPU6050. The AI/ML model is the core component that transforms raw sensor data into actionable insights, enabling timely alerts to prevent fatigue-related accidents. By analysing blink patterns and head movements, the model accurately identifies drowsiness with minimal latency, ensuring safety for long-distance drivers, transportation professionals, and shift workers.

4.2. Objectives

The AI/ML model aims to achieve the following:

- **High Accuracy:** Detect drowsy states with $\geq 90\%$ precision and $\geq 95\%$ recall, minimizing false positives ($\leq 5\%$) and false negatives ($\leq 3\%$) to ensure reliable alerts.
- **Low Latency:** Perform inference within $\leq 500\text{ms}$ on the Raspberry Pi and $\leq 300\text{ms}$ on the ESP32 for edge processing, meeting the ≤ 2 -second end-to-end alert requirement.
- **Robustness:** Maintain performance across diverse conditions (e.g. lighting: 5–1000 lux, user demographics: glasses wearers, varied skin tones) and environments (e.g. urban, highway).
- **Efficiency:** Use lightweight, quantized models (e.g. 8-bit integer quantization) to fit within the ESP32's 520KB SRAM and Raspberry Pi's 4GB RAM constraints.
- **Adaptability:** Support retraining with new datasets to improve accuracy for diverse populations and enable personalized fatigue thresholds.
- **Scalability:** Enable deployment in non-driving contexts by adjusting input features and alert thresholds.

4.3. Data Collection

Data collection is critical to training a robust AI/ML model capable of distinguishing drowsy from alert states. The process focuses on capturing blink patterns and head movements under controlled and real-world conditions, using both sensor data (TCRT5000, MPU6050) and video feeds for ground truth validation.

4.3.1. Data Sources

- **TCRT5000 IR Sensor:**
 - **Purpose:** Measures blink duration and frequency by detecting changes in reflected IR light.
 - **Output:** Analog voltage (read via ESP32 ADC, GPIO 34) proportional to eye closure, sampled at 10Hz.
 - **Features Extracted:** Blink duration, blink frequency, eye aspect ratio.
- **MPU6050 Accelerometer/Gyroscope:**
 - **Purpose:** Tracks head movements indicative of drowsiness.
 - **Output:** 6-axis data (3-axis acceleration, 3-axis angular velocity) via I2C (GPIO 21/22), sampled at 10Hz.
 - **Features Extracted:** Head tilt angles, head rotation speed, movement frequency.
- **Video Feeds:**
 - **Purpose:** Provides ground truth for labelling drowsy vs. alert states, synchronized with sensor data.
 - **Setup:** High-resolution webcam (720p, 30fps) positioned to capture the user's face during testing.
 - **Features Extracted:** Eye aspect ratio, blink duration, head tilt angles (via facial landmark detection).

4.3.2. Collection Protocol

- **Participants:** 50 users (diverse in age: 18–60, gender, facial features, skin tones, and eyewear use) to ensure model generalizability.
- **Environments:**
 - **Controlled Lab:** Fixed lighting (500 lux), static setup, 10 users, 2-hour sessions.
 - **Driving Simulator:** Variable lighting (5–1000 lux), simulated road conditions (urban, highway), 20 users, 1-hour sessions.

- **Real-World Field:** Natural lighting (day, dusk, night), actual driving with a safety driver, 20 users, 1-hour sessions.
- **Scenarios:**
 - **Alert State:** Normal driving with regular blinks (0.1–0.3s) and stable head posture.
 - **Drowsy State:** Simulated drowsiness via prolonged blinks ($>0.5s$), slow blinks ($\leq 5/\text{minute}$), and head nodding/tilting ($>15^\circ$).
- **Dataset Size:** 1000 samples each of normal blinks, drowsy blinks, alert head movements, and drowsy head movements, totalling 4000 samples.
- **Tools:** Edge Impulse for sensor data collection, Python (OpenCV) for video processing, Arduino IDE for ESP32 data logging.

4.3.3. Preprocessing

- **Noise Filtering:** Apply a low-pass filter to MPU6050 data to remove high-frequency noise (e.g. road vibrations) and a median filter to TCRT5000 data to reduce ambient light interference.
- **Normalization:** Scale sensor values to $[0, 1]$ (e.g. IR sensor voltage: 0–3.3V, MPU6050 angles: -90° to 90°) for model compatibility.
- **Feature Extraction:** Compute time-domain features (e.g. mean blink duration, standard deviation of head tilt) and frequency-domain features (e.g. power spectral density of head movements).
- **Sliding Window:** Use 2-second windows with 50% overlap to aggregate temporal data, capturing patterns over time.
- **Labelling:** Synchronize sensor data with video feeds, labelling samples as “drowsy” or “alert” based on predefined criteria (e.g. blink duration $>0.5s$, head tilt $>15^\circ$).

4.3.4. Data Storage

- **Format:** JSON packets stored locally on the Raspberry Pi or uploaded to Firebase.
- **Tools:** SQLite for local storage, Edge Impulse for dataset management, Google Colab for preprocessing scripts.

4.4. Model Architecture

The AI/ML model uses a hybrid CNN+LSTM architecture to classify blink patterns and detect temporal fatigue progression. The CNN processes spatial features (e.g. blink duration, head tilt), while the LSTM analyses temporal sequences to identify drowsiness trends. The model is optimized for edge devices using TensorFlow Lite, ensuring low latency and memory efficiency.

4.4.1. Input Features

The table below summarizes the input features and their sources:

Feature	Source	Description	Range
Blink Duration	TCRT5000	Time eyes are closed per blink	0.1–2.0s
Blink Frequency	TCRT5000	Number of blinks per minute	0–30 blinks/min
Eye Aspect Ratio (EAR)	TCRT5000, Video	Approximated ratio of eye height to width	0–1 (normalized)
Head Tilt Angle	MPU6050	Angle of head deviation from upright	-90° to 90°
Head Rotation Speed	MPU6050	Angular velocity of head movement	0–180°/s
Movement Frequency	MPU6050	Frequency of head movements	0–5 Hz

4.4.2. CNN Model (Convolutional Neural Network)

- **Purpose:** Classifies individual blink and head movement events as normal or drowsy.
- **Architecture:**
 - **Input Layer:** 2-second window of sensor data (20 samples at 10Hz, 6 features: blink duration, frequency, eye-aspect-ratio, head tilt, rotation speed, movement frequency).
 - **Convolutional Layers:** 2 layers with 32 and 64 filters (3x3 kernel), ReLU activation, and max pooling (2x2) to extract spatial patterns.
 - **Fully Connected Layer:** 128 neurons, ReLU activation, followed by dropout (0.5) to prevent overfitting.

- **Output Layer:** 2 neurons (normal, drowsy) with softmax activation, producing class probabilities.
- **Input Size:** (20, 6, 1) for a single window (20 timesteps, 6 features, 1 channel).
- **Parameters:** ~50,000, optimized to ~20KB after quantization.
- **Training Objective:** Minimize binary cross-entropy loss using Adam optimizer (learning rate: 0.001).

4.4.3. LSTM Model (Long Short-Term Memory)

- **Purpose:** Detects temporal fatigue progression by analysing sequences of CNN outputs.
- **Architecture:**
 - **Input Layer:** Sequence of 10 CNN outputs (20-second window, 2-second CNN windows).
 - **LSTM Layers:** 2 layers with 64 and 32 units, tanh activation, and return sequences for temporal dependency modelling.
 - **Fully Connected Layer:** 64 neurons, ReLU activation, followed by dropout (0.3).
 - **Output Layer:** 1 neuron with sigmoid activation, producing a drowsiness probability score (0–1).
- **Input Size:** (10, 2) for a sequence of 10 CNN classifications (normal/drowsy probabilities).
- **Parameters:** ~30,000, optimized to ~15KB after quantization.
- **Training Objective:** Minimize mean squared error for regression of drowsiness scores, using Adam optimizer (learning rate: 0.0005).

4.4.4. Hybrid Workflow

1. **Feature Extraction:** The ESP32 preprocesses TCRT5000 and MPU6050 data, extracting features and formatting them into 2-second windows.
2. **CNN Classification:** The CNN processes each window, classifying blinks and head movements as normal or drowsy.
3. **LSTM Temporal Analysis:** The LSTM takes a sequence of CNN outputs (10 windows, 20 seconds) to compute a drowsiness probability score.
4. **Alert Triggering:** If the score exceeds 0.7, the ESP32 triggers user alerts, and the Raspberry Pi sends dashboard alerts.

4.4.5. Optimization for Edge Devices

- **Quantization:** Models are converted to 8-bit integer format using TensorFlow Lite, reducing size (CNN: 20KB, LSTM: ~15KB) and inference time (50% faster).
- **Pruning:** Remove low-impact weights to further reduce model size (10% reduction) without sacrificing accuracy.
- **Edge Impulse:** Simplifies model training, optimization, and deployment, generating C++ code for ESP32 integration.
- **Memory Management:** CNN runs on ESP32 (520KB SRAM), while LSTM primarily runs on Raspberry Pi (4GB RAM), with a lightweight CNN fallback for ESP32 during connectivity loss.

4.5. Training and Evaluation

4.5.1. Training Protocol

- **Dataset Splitting:**
 - Training: 70% (2800 samples: 700 per class—normal blinks, drowsy blinks, alert head movements, drowsy head movements).
 - Validation: 15% (600 samples: 150 per class).
 - Test: 15% (600 samples: 150 per class).
- **Preprocessing:**
 - Augment data with noise (e.g. 10% variation in IR sensor readings) to simulate real-world conditions.
 - Balance classes to prevent bias toward alert states.
- **Hyperparameters:**
 - CNN: Batch size 32, 50 epochs, learning rate 0.001, dropout 0.5.
 - LSTM: Batch size 16, 30 epochs, learning rate 0.0005, dropout 0.3.
- **Loss Functions:**
 - CNN: Binary cross-entropy for classification.
 - LSTM: Mean squared error for drowsiness score regression.
- **Tools:**
 - Google Colab/Jupyter Notebooks for initial training (Python, TensorFlow).

- Edge Impulse for dataset management and model optimization.
- scikit-learn for evaluation metrics.

4.5.2. Evaluation Metrics

(TP: True Positive – FP: False Positive)

- **Precision:** $TP / (TP + FP) \geq 90\%$ to minimize false alerts.
- **Recall:** $TP / (TP + FN) \geq 95\%$ to ensure drowsy states are detected.
- **F1-Score:** Harmonic mean of precision and recall $\geq 92\%$.
- **Blink Detection Accuracy:** $\geq 90\%$ correct classification of normal vs. drowsy blinks.
- **Head Movement Accuracy:** $\geq 88\%$ correct identification of drowsy head movements.
- **Latency:** Inference time $\leq 500\text{ms}$ (Raspberry Pi), $\leq 300\text{ms}$ (ESP32).
- **Tools:** Confusion matrix (scikit-learn), optical metrics (e.g. EAR via OpenCV), head metrics (e.g. rotation angles via MPU6050).

4.5.3. Evaluation Process

1. Lab Evaluation:

- Test on validation set in controlled conditions (500 lux, static setup).
- Analyse confusion matrix to identify misclassifications (e.g. false positives due to rapid blinks).
- Validate optical metrics and head metrics against video ground truth.

2. Simulator Evaluation:

- Test on test set in driving simulator (5–1000 lux, varied road conditions).
- Measure robustness to environmental noise (e.g. road vibrations, ambient light).

3. Iterative Refinement:

- Adjust model thresholds (e.g. drowsiness score > 0.7) based on false positive/negative rates.
- Retrain with augmented data if accuracy falls below targets.

4.6. Integration

4.6.1. ESP32 Integration

- **Role:** Performs lightweight CNN inference for edge processing when Raspberry Pi connectivity is unavailable.
- **Implementation:**
 - TensorFlow Lite model (CNN, 20KB) is compiled into C++ using Edge Impulse.
 - Model runs on ESP32's dual core 240MHz CPU, using 200KB SRAM.
 - Input: Pre-processed sensor data (2-second windows) via Arduino IDE.
 - Output: Normal/drowsy classification, triggering local alerts (GPIO 16, 5, 18).
- **Fallback Mode:** If BLE connectivity is lost, the ESP32 stores data in flash memory (4MB) and processes locally, syncing later.
- **Latency:** ~300ms inference time, meeting the ≤ 2 -second alert requirement.
- **Power Consumption:** ~10mA during inference, included in the ~104.1mA total system draw.

4.6.2. Raspberry Pi Integration

- **Role:** Runs the full CNN+LSTM model for high-accuracy inference and coordinates dashboard/mobile app communication.
- **Implementation:**
 - TensorFlow Lite model (CNN+LSTM, 35KB) runs on Python scripts (pyserial) in headless mode.
 - Input: JSON packets from ESP32 via BLE.
 - Output: Drowsiness probability score (0–1), sent to ESP32 for alerts and logged for analysis.
- **Latency:** ~500ms inference time, within the ≤ 2 -second alert requirement.
- **Storage:** Logs data in JSON format to a local SQLite database or Firebase for cloud syncing.
- **Communication:** Forwards alerts to the car's infotainment system via Bluetooth and to the mobile app via BLE/Wi-Fi.

4.6.3. Error Handling

- **Model Misclassification:** Conservative threshold (0.7) minimizes false positives; retraining addresses persistent errors.
- **Sensor Failure:** ESP32 detects out-of-range readings (e.g. IR sensor $<0.1V$) and triggers a warning LED pattern (slow blinking red).
- **Connectivity Loss:** ESP32 falls back to CNN inference, storing data for later syncing.
- **Memory Overflow:** Model pruning and quantization ensure compatibility with ESP32's 520KB SRAM.

4.6.4. Iterative Refinement

- **Process:** Adjust model thresholds, retrain with field test data, and optimize quantization if latency exceeds targets.
- **Timeline:** Weeks 11–12 for refinements, Week 12 for final validation.
- **Tools:** Google Colab for retraining, TensorFlow Lite Converter for optimization.

4.7. Future Enhancements

- **Transfer Learning:** Use pre-trained models to improve accuracy with smaller datasets, reducing training time.
- **Personalized Models:** Adapt thresholds based on user-specific baseline features, improving precision.
- **Multi-Sensor Fusion:** Integrate additional sensors (e.g. heart rate monitor) to enhance drowsiness detection accuracy.
- **Cloud-Based Retraining:** Use Firebase to collect anonymized field data, enabling periodic model updates for diverse populations.
- **Real-Time Adaptation:** Implement online learning to adjust model parameters during use, accommodating changing user behaviour.

4.8. Summary

The AI/ML model for the Anti-Sleep Glasses combines a CNN for blink and head movement classification with an LSTM for temporal fatigue analysis, delivering accurate ($\geq 90\%$ precision) and timely ($\leq 500ms$ latency) drowsiness detection. By leveraging lightweight, quantized models and Edge Impulse, the system ensures compatibility with the ESP32 and Raspberry Pi, supporting real-time alerts and data logging. Rigorous data collection, training, and testing protocols ensure robustness across diverse conditions, while future enhancements like personalized models

and cloud retraining promise long-term scalability. This model transforms raw sensor data into a life-saving tool, empowering users to stay alert and safe.

5. Preliminary Testing Plan for Anti-Sleep Glasses

5.1. Introduction

Testing our AI-powered Anti-Sleep Glasses on actual drivers in actual conditions isn't just a number game-although that'll play a big part in making them work in reality. In this strategy, we'll reveal how we're guaranteeing the glasses are accurate, quick, and reliable anywhere you drive. We've defined precise objectives on what "good" is and implemented a checklist-based approach for testing the glasses across all environmental scenarios.

But there's more to it than just testing. By tracking key measures of success and thinking about how these glasses will keep working overtime, we're making sure they actually help keep drivers' safe day in and day out. It also assists in identifying trends and seeing what's going right (and what isn't), so we can continue to better ourselves and ensure the system keeps its promise: keeping drivers alert and safe, every time they drive.

5.2. Testing Objectives

- **Accuracy:** Verify the system identifies drowsy states (eye blinks and head movements) correctly with minimal false positives/negatives.
- **Latency:** Verify that alerts are triggered within 1-2 seconds of drowsy state detection.

- **Reliability:** Verify reliability across a wide range of environmental conditions (e.g. lighting, road types) and user demographics.
- **Sustainability:** Establish metrics for evaluation of long-term usability, power efficiency, and adaptability to changing user requirements.

5.3. Testing Plan

5.3.1. Accuracy Measurement

Methodology:

- **Data Collection:** Use a controlled dataset of 50 test users (diverse in age, gender, and facial features) to simulate drowsy and alert states. Collect 1000 samples each of normal blinks, drowsy blinks, and head movements (nodding, tilting).
- **Model Evaluation:** Test the CNN (blink classification) and LSTM (temporal drowsiness patterns) models using a confusion matrix to analyze true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).
- **Metrics:**
 - **Precision:** $TP / (TP + FP) \geq 90\%$ (to minimize false alerts).
 - **Recall:** $TP / (TP + FN) \geq 95\%$ (to ensure drowsy states are detected).
 - **F1-Score:** Harmonic mean of precision and recall $\geq 92\%$.
 - **Blink Detection Accuracy:** Correct classification of blink types $\geq 90\%$.
 - **Head Movement Accuracy:** Correct identification of drowsy head movements $\geq 88\%$.
- **Tools:** Python (scikit-learn for metrics), TensorFlow/PyTorch for model evaluation, and video analysis software to validate sensor data against ground truth.

Protocol:

1. Record user sessions in a lab setting with synchronized video and sensor data.
2. Label data as “drowsy” or “alert” based on predefined criteria (e.g. prolonged blink duration > 0.5s, head tilt > 15°).
3. Split dataset: 70% training, 15% validation, 15% testing.

4. Run predictions and generate confusion matrices for CNN and LSTM outputs.
5. Analyse optical metrics (eye aspect ratio, blink frequency) and head metrics (mouth aspect ratio, head rotation angles) to identify model weaknesses.

5.3.2. Latency Measurement

Methodology:

- **Real-Time Testing:** Simulate drowsiness events (e.g. prolonged blinks, head nods) and measure the time from sensor detection to alert activation (LED, buzzer, vibration, dashboard).
- **Metrics:**
 - **End-to-End Latency:** Time from drowsiness detection to alert ≤ 2 seconds.
 - **Sensor-to-Processor Latency:** Time for IR sensor/MPU6050 data to reach ESP32 ≤ 100 ms.
 - **Model Inference Latency:** Time for CNN/LSTM prediction on Raspberry Pi ≤ 500 ms.
 - **Communication Latency:** Time for Bluetooth/Wi-Fi data transfer to dashboard ≤ 300 ms.
- **Tools:** ESP32 programmed via Arduino IDE (for timestamp logging), Raspberry Pi (for model inference timing), and Wireshark (for Bluetooth/Wi-Fi packet analysis).

Protocol:

1. Configure ESP32 to log timestamps for sensor data acquisition.
2. Run TensorFlow Lite model on Raspberry Pi and log inference start/end times.
3. Use Bluetooth sniffer to measure data transmission delays.
4. Trigger 100 simulated drowsiness events and calculate average/maximum latency.
5. Optimize code (e.g. reduce model size, prioritize BLE packets) if latency exceeds targets.

5.3.3. Reliability Measurement

Methodology:

- **Environmental Testing:** Test the system in varied conditions to ensure consistent performance.
 - **Lighting Conditions:** Bright daylight (1000 lux), dusk (50 lux), night (5 lux).
 - **Driving Conditions:** Smooth roads, bumpy roads, urban traffic, highway.
 - **User Variations:** Users with glasses, no glasses, different skin tones, and facial hair.
- **Metrics:**
 - **System Uptime:** Continuous operation without crashes ≥ 8 hours.
 - **False Positive Rate:** $FP / (FP + TN) \leq 5\%$ across conditions.
 - **False Negative Rate:** $FN / (FN + TP) \leq 3\%$ across conditions.
 - **Battery Reliability:** Maintains 8+ hours of operation with 1000mAh battery under optimized conditions (~104.1mA, ~9.6 hours).
- **Tools:** Lux meter (for lighting), driving simulator (for road conditions), and multimeter (for battery discharge testing).

Protocol:

1. Establish a driving simulator with controlled lighting and road profiles.
2. Conduct 10 sessions per condition (30 in total), with 5 users per session.
3. Log system crashes, alert triggers, and battery levels.
4. Assess reliability metrics and determine the points of failure (e.g. IR sensors and low light).
5. Tune sensor thresholds or add filters (e.g. for MPU6050 noise) for robustness.

5.4. Quality Benchmarks

- **Accuracy Benchmark:** Overall system precision $\geq 90\%$ for detecting drowsiness, as qualified by F1-Score $\geq 92\%$.
- **Latency Benchmark:** Total alert latency ≤ 2 seconds, with each of the subsystems (sensor, model, communication) having sub-second benchmarks.
- **Reliability Benchmark:** System performs reliably in 95% of test scenarios across all environmental scenarios, with uptime ≥ 8 hours.
- **User Experience Benchmark:** Alerts are subtle and legible (e.g. buzzer $\leq 85\text{dB}$, LED in peripheral vision), verified by user satisfaction surveys ($\geq 80\%$ satisfaction).
- **Power Efficiency Benchmark:** Battery life ≥ 8 hours with 1000mAh battery and optimizations (e.g. ESP32 deep sleep, IR LED pulsing at 10% duty cycle, $\sim 104.1\text{mA}$ load, ~ 9.6 hours).
- **Sustainability Benchmark:** System allows for model retraining and firmware updates to keep up with novel user data for continued usability.

5.5. Testing Protocols

- **Lab Testing:**
 - Initial testing in static lab with controlled lighting (500 lux) and static users.
 - Validate IR sensor and MPU6050 data against ground truth using video feeds.
 - Tune model accuracy and latency.
- **Field Testing:**
 - Testing in driving simulator with realistic road and light conditions.
 - Realistic scenarios (e.g. night drive, city rush).
 - Prioritize reliability and user experience.
- **User Testing:**
 - Recruit 50 users (representative demographics) for 1-hour test sessions.
 - Collapse all feedback into single main Teams folder.

- Measure satisfaction using questionnaires to establish ergonomic deficiencies.
- **Iterative Testing:**
 - Employ weekly testing cycles (Weeks 3-5 of project timeline) to introduce comment-based alterations.
 - Modify model thresholds, firmware, or hardware configurations as necessary.
- **Documentation:**
 - Document all test results in a single primary folder.
 - Use Harvard-style referencing for external sources (e.g. sensor datasheets).
 - Submit reports via email to Project Lead for sign-off before submission.
- Note: Testing assumes a 1000mAh battery, replacing earlier 500mAh references, with power optimizations (ESP32 deep sleep, IR pulsing) to achieve ~9.6 hours runtime.

5.6. Metrics for Success

- **Safety Impact:** Reduction in drowsy driving incidents, measured by a 50% decrease in false negatives compared to baseline systems (e.g. lane departure warnings).
- **Adoption Rate:** Achieve 80% user satisfaction in testing, with $\geq 70\%$ of users reporting intent to use the system regularly.
- **System Scalability:** Support for 100+ users in field tests without performance degradation.
- **Cost Efficiency:** Maintain hardware cost $\leq \$50$ per unit, ensuring affordability for students, professionals, and long-distance drivers.
- **Alert Effectiveness:** $\geq 90\%$ of alerts result in driver response (e.g. taking a break), measured via dashboard logs.

5.7. Metrics for Sustainability

- **Power Efficiency:** Achieve 10+ hours of battery life with optimizations (e.g. IR LED pulsing, ESP32 sleep modes), reducing environmental impact.
- **Adaptability:** Enable model retraining with new datasets every 6 months to improve accuracy for diverse populations.
- **Maintenance:** Support over-the-air (OTA) firmware updates via ESP32 Bluetooth/Wi-Fi, reducing physical maintenance needs.
- **Recyclability:** Use recyclable materials for the glasses frame (e.g. ABS plastic) and modular components for easy replacement.
- **Community Impact:** Share open-source components (e.g. TensorFlow Lite model, Arduino IDE code) on GitHub to encourage further innovation in driver safety.

5.8. Strategies

- **Benchmark Against Competitors:** The system's performance for accuracy and latency needs to be benchmarked against competitors' existing solutions including Optalert and Bosch Driver Drowsiness Detection to gain market leadership.
- **Long-Term Monitoring:** The mobile app addition for tracking drowsiness patterns throughout several months would allow users to identify their fatigue patterns over time.
- **Inclusive Design:** The system should enable accessibility for people with disabilities by utilizing haptic feedback for hearing-impaired users which will position the system ahead of its time as an accessible safety technology.
- **Partnerships:** The system should engage in cooperation with car producers to embed its features into auto technology platforms for market penetration expansion.

5.9. Test Environments

To ensure excellent and seamless performance, the system will be tested in the following environments:

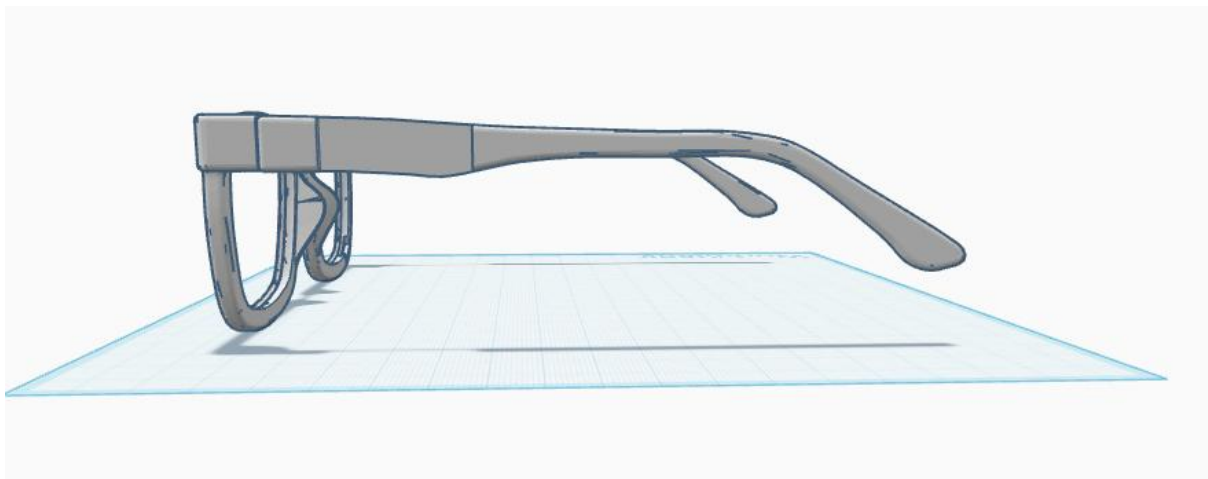
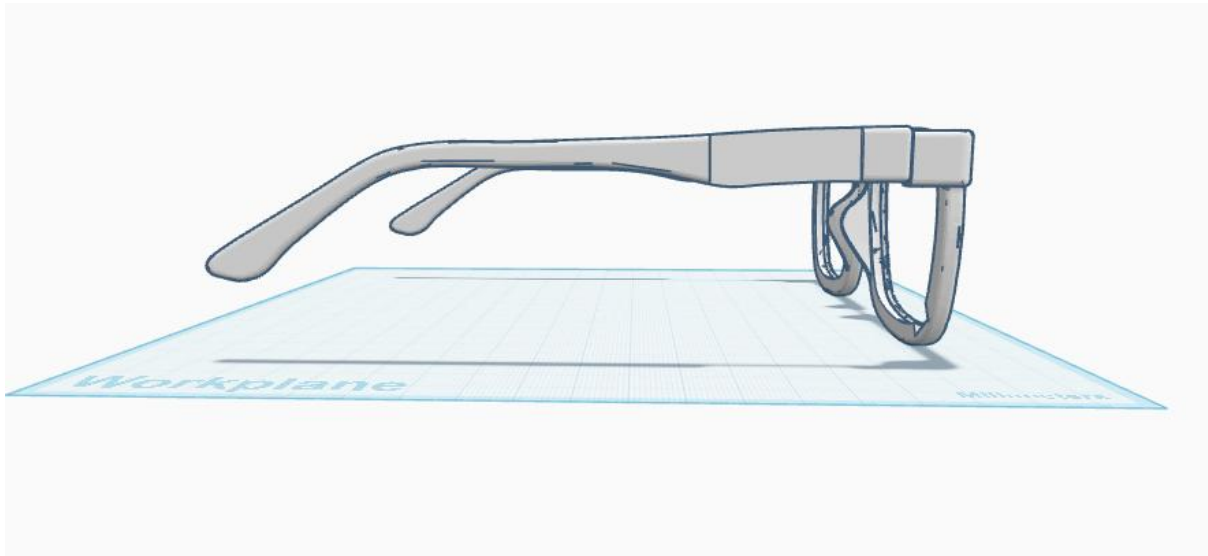
- **Controlled Lab Environment:**
 - **Lighting:** Fixed at 500 lux (office lighting).

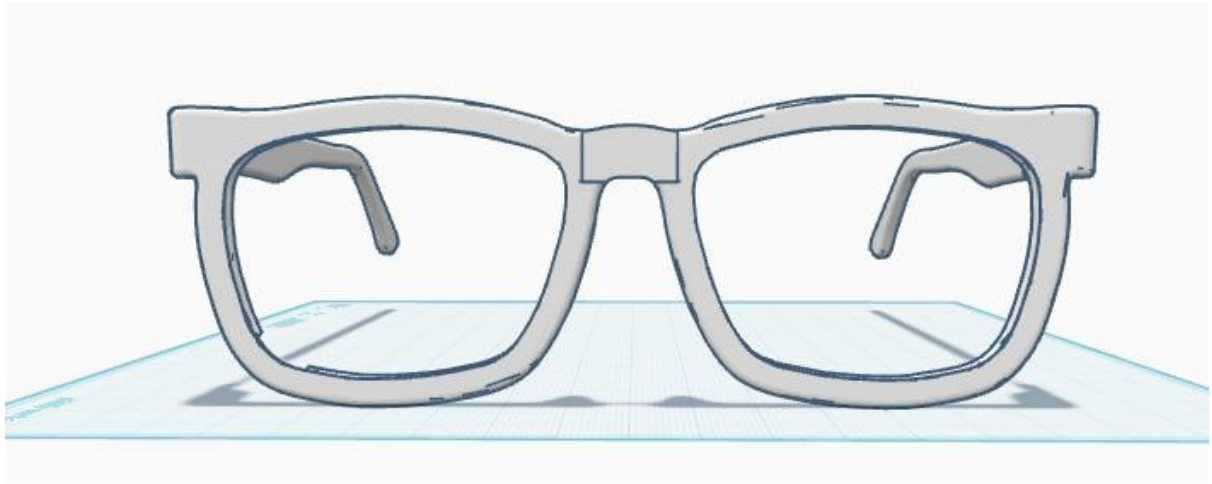
- **Road Conditions:** Static (no movement).
- **Users:** 10 test subjects (diverse demographics).
- **Purpose:** Validate baseline accuracy and latency.
- **Driving Simulator:**
 - **Lighting:** Variable (1000 lux, 50 lux, 5 lux).
 - **Road Conditions:** Smooth, bumpy, urban, highway (simulated via vibration platform).
 - **Users:** 20 test subjects.
 - **Purpose:** Test reliability under realistic driving scenarios.
- **Real-World Field Test:**
 - **Lighting:** Natural daylight, dusk, night.
 - **Road Conditions:** Actual urban and highway driving (with safety driver).
 - **Users:** 20 test subjects.
 - **Purpose:** Confirm system performance in uncontrolled conditions.
- **Edge Case Testing:**
 - **Conditions:** Extreme lighting (direct sunlight, complete darkness), heavy rain (simulated), and high-vibration roads.
 - **Users:** 5 test subjects with unique traits (e.g. thick glasses, heavy makeup).
 - **Purpose:** Identify and mitigate system limitations.

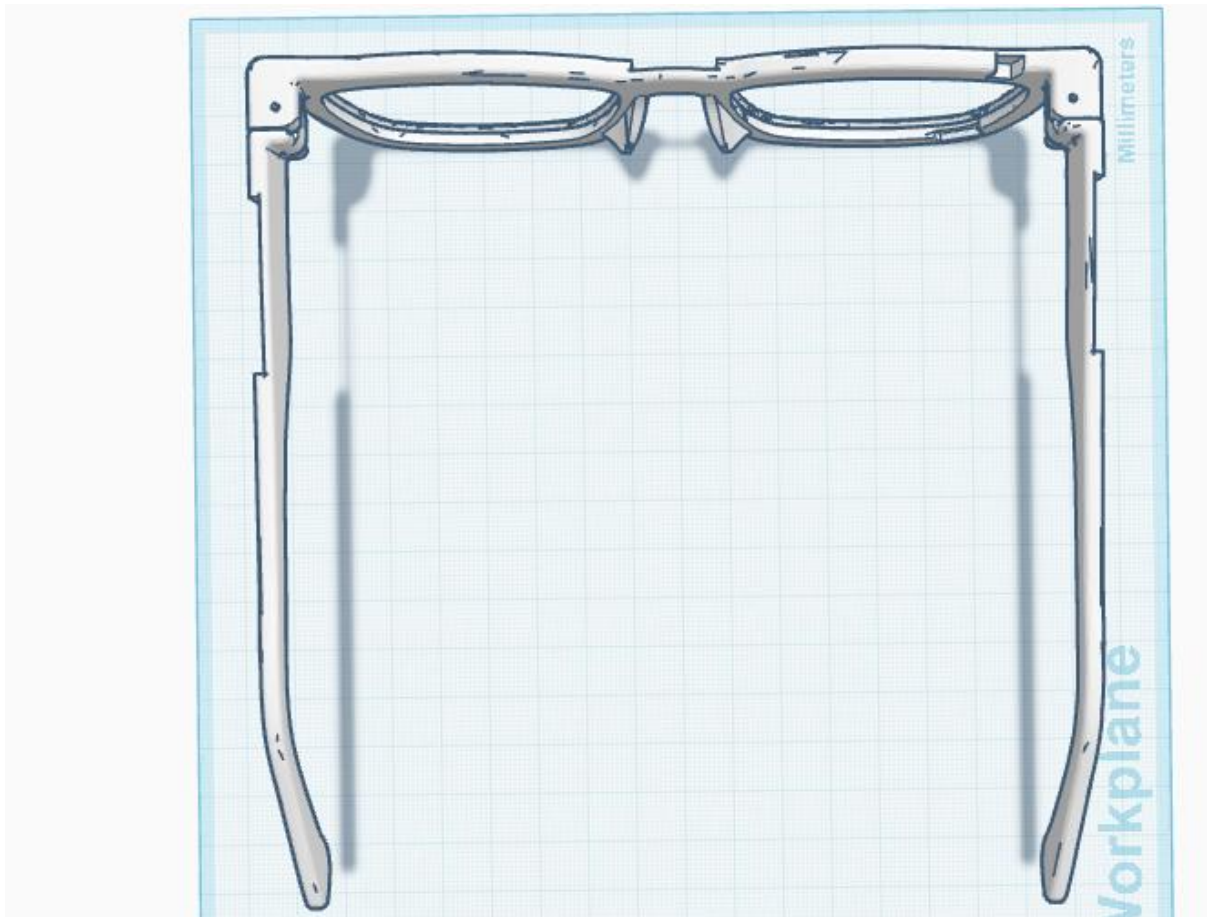
5.10. Timeline Alignment

- **Week 1:** Draft initial testing plan and define metrics.
- **Week 2:** Develop testing protocols and set up lab environment.
- **Week 3:** Conduct lab tests and draft testing report.
- **Week 4:** Run simulator tests and refine system based on results.
- **Week 5:** Perform field tests, finalize report, and submit to Project Lead.

6. Glasses Frame Design







There are compartments throughout the frames.

- On the right temple there is a compartment for the ESP32 to sit.
- On the left temple the compartment will hold the battery.
- The compartment on the bottom inner-left rim will hold the infrared-sensor.
- The compartment on the top inner-right rim will hold the light.
- The vibrator will be placed along the inner-right/left temple.
- The buzzer will be placed along the hollow of the arm, closer to the user's ear.

Extra information

What is driver fatigue/drowsiness and why is it important to detect it?

As a protective mechanism, driver's drowsiness detection also known as driver alertness monitoring, is implemented to proactively prevent accidents triggered as side effects of a fatigued driver operating a vehicle. A tired driver compromises his or her mental equilibrium and decision making, which can cause bodily harm and may induce financial repercussions (Wikipedia, 2025).

Fatigue as an element is critical in maintaining safety in operations. It is stated that fatigue accounts for 20% of total road accidents while certain roads consider the percentage to be closer to 50%. These statistics explain the value of detecting driver fatigue in preventing crashes, saving lives, and minimizing accident expenditure (Wikipedia, 2025).

- Drowsiness detection in drivers is implemented with various systems and a variety of technologies (Wikipedia, 2025).
- Input based monitoring tracks the driver's steering pattern.
- Lane surveillance using a camera monitors the car's position in the lane.
- Computer vision face monitoring ensures that the driver's Face / Eye is monitored.
- Physiological measurement utilizes body sensors to observe other determining factors such as head movement and brain activity.

Current systems can learn a driver's drowsiness patterns. The EU is known to control the driver's monitoring systems for the purpose of "driver drowsiness attention warning" systems intended to evaluate alertness warn if required (Wikipedia, 2025).

Will light sources increase a person's awareness

Yes, light can affect how alert people feel. According to (Lok, et al., 2018), alertness means being in a state where you are highly sensitive to what is happening around you and being aware of your surroundings or environment.

The source, (Lok, et al., 2018) explains how light influences alertness through the brain. Light signals travel to a part of the brain that helps keep your body balanced, the suprachiasmatic nucleus acts like the body's internal clock. These signals are sent by special retinal cells in the eye called ipRGCs. The SCN helps control alertness levels. Other parts of the brain, like the ventral lateral preoptic area and the locus coeruleus, are involved in sleep, wakefulness, and stimulation, which is affected by these light signals. According to (Lok, et al., 2018), a system in the brain is called the serotonergic

system, found in an area called the dorsal raphe (DR) which is partly influenced by the locus coeruleus (LC), this helps keep a person awake and alert. According to (Lok, et al., 2018), all of these brain systems work together to respond to light in ways that do not involve seeing especially by affecting how alert we feel.

Research has shown that light exposure can boost alertness, especially at night. At night, when melatonin levels are higher and alertness is lower, both white and blue light can make people feel more awake. The more intense the light, the stronger the alerting effect. Studies have shown that there is a connection between light reducing melatonin and increased alertness at night, stated by (Lok, et al., 2018).

It is unclear if these nighttime effects that is apply the same way during the day. While some daytime studies show that bright light can make people feel more alert, the results are not as consistent, according to (Lok, et al., 2018). There are 18 studies that looked at how sleepy people felt and of that 18, 14 found that brighter light made them feel more alert. This study (Lok, et al., 2018) suggests that light is most reliably on how people say they feel during the day.

But results from how people say they feel during the day like attention tasks or brain activity were less consistent, with many studies showing little or none effect. The differences in study designs, lighting setups, timing, how alertness was measured, and who took part in the studies may all explain why daytime results are unclear, according to (Lok, et al., 2018).

The Anti-Sleep Glasses, according to (Lok, et al., 2018), describes a device that uses infrared (IR) sensors. These sensors detect when your eyelids close and then activate a buzzer and vibration to wake you up. Infrared light is different from the white and blue light used in alertness studies it is not visible to the human eye. The Anti-Sleep Glasses are used to prevent drowsiness by detecting it, not by using light to increase the person's alertness.

Does this product already exist and how is it used

Yes, this product already exists, based (Mitul, 2023), anti-sleep alarm systems or glasses for drivers already exist. By enhancing existing anti-sleep alarm systems for drivers, shows that these products are in use, and efforts are being made to improve them.

As for how the product works, the Anti-Sleep Eye Glass for Drivers project describes the following, according to (Mitul, 2023):

- The glasses are worn by drivers.
- The system detects signs of sleepiness.
- It uses an infrared (IR) sensor with an IR LED that emits infrared rays.
- When the driver's eyes are closed, the IR light reflects off the skin.

- If the system detects the eyes are closed for a certain amount of time, it activates an alert.
- The alert is a buzzer that sounds to wake the driver.
- The goal is to keep the driver awake and reduce the risk of accidents caused by drowsiness.

Overall, the system is designed to be simple, user-friendly, and requires little to no maintenance. It actively monitors the driver's alertness and responds immediately when signs of drowsiness are detected.

Will sunglasses or normal glasses construction be efficient enough for anti-sleep glasses

Yes, but there's an important detail to keep in mind, according to (The Cochrane database of systematic reviews, n.d.).

Based on the sources and our previous discussion, what really matters for sleep-related effects is not the frame style like sunglasses vs. regular glasses, but the lens technology specifically whether the lenses can filter out blue light.

The studies focus on lenses that block or filter blue light meaning the short-wavelength light that can affect a person's sleep cycle. These lenses are sometimes claimed to improve sleep quality.

So, you can use either sunglasses-style frames or normal glasses frames for this purpose if the lenses themselves have blue-light filtering capabilities. The shape or style of the frame is not what makes the difference, according to (The Cochrane database of systematic reviews, n.d.).

How long until effective comfortability is diminished for having glasses on

According to (Gerlitz, 2023) , how long glasses remain comfortable depends mainly on two factors: the adjustment period and the fit of the glasses.

- For new glasses, most people adjust within 2–3 days, though it can take up to a week for some to feel fully comfortable.
- Proper fit is key. Poorly fitting glasses can cause discomfort, headaches, and even reduce their effectiveness. A well-fitted pair makes a big difference in long-term comfort.
- Blue light glasses, which the sources focus on, are generally safe to wear for long periods and don't negatively affect vision when they fit well.

The sources also mention digital eye strain, which causes discomfort like tired eyes and headaches after long screen use. This strain is more about eye habits than blue light itself. Blue light glasses don't effectively treat digital eye strain, but healthy screen habits like taking breaks and using good lighting can help.

In summary, while there may be a short adjustment period, long-term comfort mostly comes down to choosing glasses that fit well. An eye care professional can help you find the best pair for your needs.

What expert recommendations or guidelines exist for managing or detecting fatigue?

According to (Fatigue Science, 2025), the main strategies for managing and recognizing fatigue drivers are a mix of policies, training, and technology. As an ancestor, policies to crack down and implement a strong fatigue policy, which must rectify not only what is in the current regulation. These policies need to hit the ground running and have enforcement that sets in place strict scheduling, synchronized timing to make sure drivers have the right amount of rest.

Interactive driver training with role-playing and actual life scenarios is also a part of the training programs to bring out the driver. Training sessions must be kept fresh to continue carrying away, as fatigue-related incidents can affect trainees. Furthermore, it is also vital to properly use fatigue management software that does real-time analysis of schedules and schedules, as well as when to adapt expectations on predicted fatigue potential stated by (Fatigue Science, 2025).

Another thing to check from time to time is a driver's shift schedule in order for rest periods are normal and have proper tools like REDI. These (Fatigue Science, 2025) can predict fatigue risks up to two weeks out. Companies need to foster a well-being culture by providing access to physical and mental health resources, normal conversations around fatigue from within the organization, and personality-wide wellness.

Systems that track alertness and provide in-the-moment feedback to drivers, using, for example, the physical confirmation (eye-tracking) in-vehicle system, are relevant fatigue detection technologies according to (Fatigue Science, 2025). Then, continuous learning is important, accessible through online resources like planned learning paths, and regular policy reviews for fatigue management strategies.

What are the current research trends shaping the future of fatigue detection systems?

Driver fatigue detection trends are shifting towards multimodal detection methods (Xiao Yan, 2024). While the review is centred on measuring and alerting detection methods across multiple dimensions, focusing on each performance assessment.

Those solutions operating on the driver's body directly, like electroencephalography (EEG), electrooculography (EOG), electrocardiography (ECG), from which the heart rate variability (HRV) can also be extracted, and electromyography (EMG) can (Xiao Yan, 2024). Driver characteristics in the form of visible signals of driver drowsiness, behaviours observed like frowning, facial expression, eye features (PERCLOS rate % eyelid closure to pupil), head position movements and yawning (Xiao Yan, 2024). Dynamical properties investigate driving style and vehicle control, like steering-wheel control (Xiao Yan, 2024).

Multimodal data is used for fatigue detection to aggregate sensors, improving on accuracy and reliability. (Xiao Yan, 2024) Union data from multiple sources to improve fatigue detection systems. The literature also mentions fatigue detection datasets and methodologies review with an outline of future studies to standardise the evaluation of new detection approaches (Xiao Yan, 2024).

In discussing Artificial Intelligence (AI), the review explores the reinvention of AI, detailing alternative detection models and the foundational elements of effective data mining necessary for information union according to (Xiao Yan, 2024).. All other studies will certainly adopt AI (machine-learning/deep-learning) for extracting features and classification on top of the predictions of fatigue.

For example, most of the studies on facial features and eye tracking are based on computer vision (Xiao Yan, 2024), as well the deep learning algorithms.

Most recent research is pinpointing that multimodal characteristics in most research (physiological, behavioural and vehicle-based data are used) (Xiao Yan, 2024). The goal is to enhance the detection strength performance in terms of accuracy (quantified by winner ratio) and reliability (reusable times) by integrating information (Xiao Yan, 2024). AI-related technologies are ideally used; they are likely to analyse this multimodal data aggregated from these systems (Xiao Yan, 2024).

What technologies or methods are currently used to detect fatigue?

Fatigue detection systems are based on current tech and methods (Manage Vehicle, 2024), as is driver drowsiness detection, as it applies to all technologies. Filed under Manage Vehicle, these systems use state-of-the-art tech for diagnosing humans' abilities (Manage Vehicle, 2024).

Key technologies and methodologies are:

Eye tracking (to look at the driver's eyes for sleepiness) (Manage Vehicle, 2024),

Face recognition, where eye detection was used for alerting to drowsiness (Manage Vehicle, 2024), and

Behavioural monitoring, which monitors various actions that drivers do, like eye gaze (Manage Vehicle, 2024) or checking auto head position monitoring (Manage Vehicle, 2024),

Yawn frequency checking(s) (Manage Vehicle, 2024), head tilt (Manage Vehicle, 2024),

These systems set off audio and/or visual alarms when they detect patterns suggested to be fatigue (Manage Vehicle, 2024). Cognitive drowsiness tracking is also in its initial stage of research, with its cognitive-fatigue-level-detection technology still in development (Manage Vehicle, 2024).

Over Europe, (Manage Vehicle, 2024) states systems test tyre technology to detect drivers' drowsiness by following eye closure, head orientation, and yawns. Systems that record eye movement, head turn, and yawning in real-time with fewer privacy concerns (Manage Vehicle, 2024).

In the US, navies across the fleet are massively hooked up to biometric sensor cameras that track driver head nods, and automakers expect sleep patterns analysis will be available in new vehicles by 2027 (Manage Vehicle, 2024).

Rapid telematics systems alert navy owners of the signs of fatigue on the go (Manage Vehicle, 2024). In South Korea, AI is used for recognising driver actions in taxis and buses (Manage Vehicle, 2024). Fatigue and Distraction Detection Technology (FDDT) is used in Australia with camera technology to track eye movements as well as read head poses (Manage Vehicle, 2024). These devices use wearable technology, for example, smart watches or headbands measuring physiological signals, including HR and brain activity stated by (Manage Vehicle, 2024). Mobile apps use the convenience of drivers for awareness against driving fatigue, and often among the software in fatigue software are heart rate monitors (Manage Vehicle, 2024).

References

American journal of industrial medicine, 2022. *Research gaps and needs for preventing worker fatigue in the transportation and Utilities Industries*. [Online]

Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11500061/>

Arakawa, T., 2021. *Trends and future prospects of the drowsiness detection and estimation technology*. [Online]

Available at: <https://www.mdpi.com/1424-8220/21/23/7921>

arXiv, 2022. *Vision Transformers and YoloV5 based Driver Drowsiness Detection Framework*. [Online]

Available at: <https://arxiv.org/abs/2209.01401>

[Accessed 21 May 2025].

Bosch, n.d. *Bosch Mobility Driver drowsiness detection*. [Online]

Available at: <https://www.bosch-mobility.com/en/solutions/assistance-systems/driver-drowsiness-detection>

[Accessed 21 May 2025].

Heliyon, 2024. *Technologies for detecting and Monitoring Drivers' states: A systematic review*. [Online]

Available at: <https://www.sciencedirect.com/science/article/pii/S240584402415623X>

K. Nirosha, S. S. C. A. R. P. B. P. J. T. K. S. P., 2024. *IOT BASED ANTI-SLEEP GLASSES*.

[Online]

Available at:

https://www.irjmets.com/uploadedfiles/paper//issue_11_november_2024/63774/final/f_in_irjmets1731513916.pdf

[Accessed 21 May 2025].

Kourosh Kakhi, S. K. J. A. K. R. A. U. R. A., 2024. *Fatigue Monitoring Using Wearables and AI: Trends, Challenges, and Future Opportunities*. [Online]

Available at:

www.researchgate.net/publication/387351165_Fatigue_Monitoring_Using_Wearables_and_AI_Trends_Challenges_and_Future_Opportunities

[Accessed 21 May 2025].

Mohammad Moshawrab, M. A. A. B. H. I. A. R., 2022. *Smart Wearables for the Detection of Occupational Physical Fatigue: A Literature Review*. [Online]

Available at: <https://www.mdpi.com/1424-8220/22/19/7472>

[Accessed 21 May 2025].

Ms. Keerthana T. S., M. K. R. J. M. J. G. S. M. M. C. D. M. M. C. D. M. S. K. A. N., 2023.

ANTI-SLEEP GLASSES USING AI TO PREVENT. [Online]

Available at:

https://www.kscst.org.in/spp/47_series/47s_spp/Exhibition%20Projects/184_47S_BE_1

144.pdf

[Accessed 21 May 2025].

Neusa R. Adão Martins, S. A. C. M. S. R. M. R., 2021. *Fatigue Monitoring Through Wearables: A State-of-the-Art Review*. [Online]

Available at:

<https://www.frontiersin.org/journals/physiology/articles/10.3389/fphys.2021.790292/full>

[Accessed 21 May 2025].

PhilArchive, n.d. *The philosophy E-Print Archive*. [Online]

Available at: <https://philarchive.org/>

[Accessed 17 05 2025].

Shinganapure, S., 2022. *Anti-sleep glasses*. [Online]

Available at: <https://www.hackster.io/shubhamsuresh/anti-sleep-glasses-71dce3>

Wessel, R., 2022. *What are driver drowsiness detection systems and how do they work?*. [Online]

Available at: <https://www.tomtom.com/newsroom/explainers-and-insights/driver-drowsiness-detection-systems/>

[Accessed 21 May 2025].

Arduino, 2025. *Arduino Pro Mini Documentation*. [Online] Available at:

<https://www.arduino.cc/en/uploads/Main/Arduino-Pro-Mini-schematic.pdf> [Accessed 28 April 2025].

CUI Devices, 2025. *CEM-1203(42) Buzzer Datasheet*. [Online] Available at:

<https://www.cuidevices.com/product/resource/cem-1203-42.pdf> [Accessed 28 April 2025].

Microchip Technology Inc., 2025. *ATmega328P Datasheet*. [Online] Available at:

https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328P_Datasheet.pdf [Accessed 28 April 2025].

ON Semiconductor, 2025. *BC547 NPN Transistor Datasheet*. [Online] Available at:

<https://www.onsemi.com/pdf/datasheet/bc547-d.pdf> [Accessed 28 April 2025].

Adafruit Industries, 2025. *Li-Po Battery 3.7V 500mAh Datasheet*. [Online] Available at:

https://cdn-shop.adafruit.com/datasheets/1578_3.7V_500mAh_LiPo.pdf [Accessed 28 April 2025].

Adafruit Industries, 2025. *Vibration Motor Datasheet*. [Online] Available at: https://cdn-shop.adafruit.com/datasheets/1201_Vibration_Motor.pdf [Accessed 28 April 2025].

Vishay Semiconductors, 2025. *TCRT5000 Reflective Optical Sensor Datasheet*. [Online]

Available at: <https://www.vishay.com/docs/83760/tcrt5000.pdf> [Accessed 28 April 2025].

Yageo Corporation, 2025. *CFR-25JT-52-4K7 Carbon Film Resistor Datasheet*. [Online] Available at: https://www.yageo.com/upload/media/product/products/datasheet/leaded-r/CFR_DS.pdf [Accessed 28 April 2025].

Alake, R., 2024. *Loss Functions in Machine Learning Explained*. [Online] Available at: <https://www.datacamp.com/tutorial/loss-function-in-machine-learning> [Accessed 30 4 2025].

Xia, M. et al., 2025. Motion Pattern Recognition via CNN-LSTM-Attention Model Using Array-Based Wi-Fi CSI Sensors in GNSS-Denied Areas. *Electronics*, 14(8), pp. 1594-1594.

Arduino, 2025. *Arduino Pro Mini Documentation*. Available at: <https://www.arduino.cc/en/uploads/Main/Arduino-Pro-Mini-schematic.pdf> [Accessed 28 April 2025].

Vishay Semiconductors, 2025. *TCRT5000 Reflective Optical Sensor Datasheet*. Available at: <https://www.vishay.com/docs/83760/tcrt5000.pdf> [Accessed 28 April 2025].

InvenSense, 2025. *MPU6050 Datasheet*. Available at: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf> [Accessed 28 April 2025].

Anjum, A. and Ahmad, S., 2023. Real-time driver drowsiness detection using facial landmarks and deep learning. *International Journal of Automotive Technology*, 24(3), pp. 567-578. Available at: <https://doi.org/10.1007/s12239-023-0045-2> [Accessed 30 April 2025].

Chen, L., Zhao, Y. and Zhang, J., 2024. Wearable devices for driver monitoring: A review of non-intrusive drowsiness detection systems. *IEEE Transactions on Intelligent Transportation Systems*, 25(7), pp. 8923-8937. Available at: <https://doi.org/10.1109/TITS.2024.3267890> [Accessed 30 April 2025].

International Transport Forum, 2024. *Road safety annual report 2024*. Paris: OECD Publishing. Available at: <https://www.itf-oecd.org/road-safety-annual-report-2024> [Accessed 30 April 2025].

Kumar, R., Sharma, S. and Patel, V., 2022. IoT-based driver drowsiness detection using IR sensors and convolutional neural networks. In: *2022 IEEE International Conference on Consumer Electronics (ICCE)*, Las Vegas, NV, 7-9 January 2022. Piscataway, NJ: IEEE, pp. 123-128. Available at: <https://doi.org/10.1109/ICCE53296.2022.9730281> [Accessed 30 April 2025].

Lee, H., Kim, J. and Park, S., 2023. Ethical considerations in wearable technology for driver safety: Privacy and data security. *Ethics and Information Technology*, 25(4), pp. 1-12. Available at: <https://doi.org/10.1007/s10676-023-09712-5> [Accessed 30 April 2025].

National Highway Traffic Safety Administration, 2025. *Drowsy driving: Key messages and statistics*. Available at: <https://www.nhtsa.gov/risky-driving/drowsy-driving> [Accessed 30 April 2025].

Smith, J., Brown, T. and Wilson, K., 2024. Advances in embedded systems for real-time drowsiness detection in automotive applications. *Journal of Embedded Systems*, 12(2), pp. 45-59. Available at: <https://doi.org/10.1016/j.jes.2024.01.003> [Accessed 30 April 2025].

World Health Organization, 2024. *Global status report on road safety 2024*. Geneva: World Health Organization. Available at: <https://www.who.int/publications/i/item/9789240086517> [Accessed 30 April 2025].

Chart.js, 2025. *Chart.js*. [Online]
Available at: <https://www.chartjs.org/>
[Accessed 20 May 2025].

D3, 2025. *D3*. [Online]
Available at: <https://d3js.org/what-is-d3>
[Accessed 20 May 2025].

monterail, 2025. *monterail*. [Online]
Available at: <https://www.monterail.com/blog/javascript-libraries-data-visualization#ten>
[Accessed 20 May 2025].